AI-Powered Incident Response System: A Rule-Based Intelligent Incident Response Expert System

A PROJECT REPORT

Submitted by

NILA ELA (241901074)

SREEDHARSHINI M (241901110)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)



RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institute

CHENNAI

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "AI-Powered Incident Response System" is the bonafide work of "Nila ELA (2116241901074) and Sreedharshini M (2116241901110)" who carried out the work under my supervision.

SIGNATURE

Dr Bhuvana S, Ph D.,

ASSISTANT PROFESSOR(SG)
Department of Computer Science and Engineering (Cyber Security)

Rajalakshmi Engineering College
Chennai - 602 105

Submitted to Project Viva-Voce Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

ABSTRACT

The AI-Powered Incident Response System is a web-based application designed to help security teams handle cybersecurity incidents such as brute-force attempts, port scans, malware, SQL injection, phishing and denial-of-service attacks. Traditional incident handling requires analysts to read, classify and rank every alert by hand, which takes significant time and depends on individual expertise. The proposed system addresses this challenge by automating incident classification, prioritization and response recommendation using classical Artificial Intelligence techniques.

Analysts report an incident with its source IP address, attack classification, number of failed attempts and target asset. The system uses a rule-based expert system with forward-chaining inference to classify the incident, heuristic evaluation to compute a priority score from 0 to 100 based on asset criticality, attack frequency and impact, and constraint satisfaction to select suitable response actions.

The application is developed using Java with Spring Boot for the backend, MySQL for database management and a web-based front end for the monitoring dashboard. The backend follows a layered architecture of controller, service, repository and entity classes, and its CRUD REST APIs were tested using Postman. Unlike approaches based on machine learning, the system uses transparent, predefined rules, so every classification can be traced to the rules that produced it.

ACKNOWLEDGEMENT

We express our sincere thanks to our beloved and honourable chairman Mr. S. Meganathan B.E., F.I.E**.,** and the chairperson Dr. M. Thangam Meganathan Ph.D., for their timely support and encouragement.

We are greatly indebted to our respected and honourable principal Dr. S.N**.** Murugesan M.E., Ph.D., for his able support and guidance.

We express our sincere thanks to Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security).

We also extend our sincere and hearty thanks to our internal guide Ms. Divya R, for her valuable guidance and motivation during the completion of this project.

We would like to extend our special gratitude to our mentor Dr Bhuvana S**,** Ph.D,Assistant Professor(SG) for her constant guidance, valuable suggestions, and continuous encouragement throughout our project journey.

Our sincere thanks to our family members, friends and other staff members of computer science engineering.

NILA ELA (241901074)

SREEDHARSHINI M (241901110)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
	LIST OF ABBREVIATIONS	**ix**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 SCOPE	3
	1.5 SECURITY REQUIREMENTS	4
	1.6 SDG ALIGNMENT	4
	1.7 ORGANIZATION OF THE REPORT	5
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	7
	2.2 RESEARCH GAPS	10
	2.3 PROPOSED SYSTEM	10
3	**SYSTEM DESIGN**	**13**
	3.1 SYSTEM ARCHITECTURE	13
	3.2 MODULE DESIGN	14
	3.3 DATA FLOW / UML DIAGRAMS	15
	3.4 THREAT MODEL AND ATTACK SURFACE	21
	3.5 SECURITY CONTROLS	23
	3.6 TOOLS AND TECHNOLOGIES	23
4	**SYSTEM DESCRIPTION**	**24**
	4.1 DATA COLLECTION	24
	4.2 DATA PREPROCESSING	24

5 CONCLUSION AND FUTURE WORK 41

5.1 CONCLUSION 41

5.2 FUTURE WORK 41

5.3 REFERENCES 42

5.4 APPENDIX

LIST OF TABLES

TABLE NO	TITLEPAGE	NO
1.1	Attack types handled by the system	1
1.2	Security requirements and the mechanism used for each	4
1.3	SDG 9 targets addressed by the project	5
2.1	Literature survey	6
2.2	Advantages and limitations of the existing system	9
2.3	Research gaps and project response	10
3.1	System modules	14
3.2	Threat model and security controls	22
3.3	Technologies used	23
4.1	Incident input attributes	24
4.2	Representative rule structure	27
4.3	Implementation status	33
4.4	API test cases executed in Postman	33
4.5	Incidents recorded during testing	34
4.6	Summary metrics reported by the system	35
4.7	Comparison of the existing approach and the proposed system	36
5.1	Core entities and fields of the database schema	44
5.2	Incident CRUD operations	44
5.3	Project schedule	44

LIST OF FIGURES

FIGURE NO	TITLEPAGE	NO
2.1	Existing incident handling workflow (manual triage process) with its limitations	9
2.2	Proposed workflow of the AI-Powered Incident Response System	11
3.1	Layered system architecture of the AI-Powered Incident Response System	13
3.2	Level 0 data flow diagram of the AI-Powered Incident Response System	16
3.3	Level 1 data flow diagram showing Modules 1 to 4 of the AI-Powered Incident Response System	17
3.4	Use case diagram of the AI-Powered Incident Response System showing the analyst, administrator and AI analysis module	18
3.5	Activity diagram of incident submission and analysis across Modules 1 to 4	19
3.6	Class diagram of the incident data model (Module 1: Incident Management and Backend)	20
3.7	Sequence diagram of the incident analysis flow (Modules 1, 2 and 3)	21
3.8	Threat model and attack surfaces of the AI-Powered Incident Response System	22
4.1	Methodology workflow of the AI analysis pipeline across Modules 1 to 4	26
4.2	Forward-chaining inference flowchart (Module 2: AI-Based Incident Analysis)	28
4.3	Heuristic priority scoring model (Module 3: Intelligent Prioritization and Response)	30
4.4	Constraint-based response selection (Module 3: Intelligent Prioritization and Response)	31
4.5	Inference trace for a brute-force incident (Modules 2 and 3)	32
4.6	Incidents by attack type in the test data (Module 4: Dashboard and Integration	34

4.7	Incidents by severity level assigned by Module 3 (Intelligent Prioritization and Response)	35
4.8	Incidents reported per day in the test data (Module 4 reports)	35
4.9	Sign-in page of the AI-Powered Incident Response System (Module 4: Dashboard and Integration)	37
4.10	Dashboard overview with summary cards, incidents over time and severity distribution (Module 4)	38
4.11	Dashboard attack types, Needs Attention list and attack-origin globe (Module 4)	38
4.12	Dashboard recent activity timeline (Module 4)	39
4.13	Report New Incident form feeding the analysis pipeline (Module 1 input with Modules 2 and 3)	39
4.14	Security incidents list with severity, priority and status (Module 1 records with Module 3 output)	40
4.15	Reports and analytics page (Module 4: Dashboard and Integration)	40

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
CVSS	Common Vulnerability Scoring System
DDoS	Distributed Denial-of-Service
DFD	Data Flow Diagram
DoS	Denial-of-Service
HTTP	Hypertext Transfer Protocol
IDPS	Intrusion Detection and Prevention System
IDS	Intrusion Detection System
IP	Internet Protocol
JSON	JavaScript Object Notation
ML	Machine Learning
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
REST	Representational State Transfer
SDG	Sustainable Development Goal
SQL	Structured Query Language
UI	User Interface
UML	Unified Modeling Language

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND

Every networked service that an organization exposes produces logs and alerts. Firewalls, intrusion detection systems, authentication servers, web servers and endpoint agents each report suspicious events, and a security team has to decide which of these events are genuine incidents, which incident must be handled first and what response is appropriate. This chapter introduces the problem, the objectives and scope of the project, its security requirements and its alignment with the Sustainable Development Goals.

1.1.1 Cybersecurity Incidents and Alert Volume

A cybersecurity incident is an event that violates or threatens the security policy of an organization. In network operations, incidents are usually first seen as alerts produced by monitoring tools. The attack types that appear most often in such alerts are summarized in Table 1.1. The same six attack classes are supported by the system developed in this project.

Table 1.1 Attack types handled by the system

Attack type	What happens	Typical target
Brute force	Repeated login attempts that guess passwords or flood an authentication service	User and administrator accounts, login APIs
Port scan	Probing a host for open ports and running services, usually before an attack	Routers, servers, firewalls
Denial-of-service (DDoS)	High-volume traffic that exhausts a service so that legitimate users cannot reach it	Public websites and APIs
Malware	Infected payload or command-and-control beacon on a host	Servers and workstations
SQL injection	Malicious query fragments inserted into application input	Databases behind web forms
Phishing	Deceptive messages or links that harvest user credentials	Employees and mailboxes

Monitoring tools generate alerts far faster than analysts can review them. Axelsson [11] showed that because genuine attacks are rare compared with normal events, even a detector with a low false-alarm rate produces many more false alarms than true detections. The

practical consequence is that security teams spend most of their time sorting alerts, and a critical incident can wait in the same queue as harmless noise.

1.1.2 Incident Response Lifecycle

The NIST Computer Security Incident Handling Guide [1] organizes incident handling into four phases, and ISO/IEC 27035-1 [7] describes a comparable process for information security incident management:

- **Preparation:** establishing the team, tools and procedures before an incident occurs.
- **Detection and analysis:** validating alerts, documenting incidents and assigning a priority to each one.
- **Containment, eradication and recovery:** limiting the damage, removing the cause and restoring normal service.
- **Post-incident activity:** reviewing the incident and improving defences.

This project concentrates on the detection and analysis phase, where incidents are classified and prioritized, and on the first decision of the containment phase, which is choosing an appropriate response. NIST guidance on intrusion detection and prevention systems [2] notes that detection products alone do not decide how an organization should respond; that decision still depends on analysts.

1.1.3 Role of Artificial Intelligence in Incident Response

Artificial intelligence offers two broad routes for supporting analysts. Data-driven methods learn patterns from labelled examples, while knowledge-based methods encode expert knowledge explicitly as rules and reason over them [5]. Expert systems built on production rules have been used since the 1970s because their conclusions can be explained step by step [6]. Threat models such as the intrusion kill chain [3] and the MITRE ATT&CK knowledge base [4] already describe attacker behaviour in a structured form that maps naturally onto rules. This project uses the knowledge-based route: incident attributes become facts, a rule base derives the classification and severity, a heuristic function computes a priority score, and constraints filter the recommended responses.

1.2 PROBLEM STATEMENT

Organizations receive a large volume of cybersecurity alerts from different sources. Manual investigation of every alert requires significant time and human effort, which makes it difficult to identify critical incidents quickly. Traditional systems detect or record incidents

but provide limited prioritization and no explainable response recommendation. The specific problems addressed are:

- Incident records, severity assessment and response guidance are spread across separate tools.
- Severity is assigned by individual judgement, so the same incident may be ranked differently by different analysts.
- Critical incidents are not separated from low-risk incidents at the time they are reported.
- Response decisions are not recorded together with the reasoning behind them.

An intelligent system is therefore required that automatically analyzes each incident, assigns a priority and recommends an appropriate response, while showing the reasoning behind each decision.

1.3 OBJECTIVES

- To develop a centralized system for collecting and managing cybersecurity incidents.
- To classify security incidents using rule-based AI techniques with forward-chaining inference.
- To determine incident severity and priority using heuristic evaluation, expressed as a score from 0 to 100.
- To recommend suitable response actions based on incident conditions and constraints.
- To provide a web-based dashboard for monitoring incidents, analytics and AI-generated recommendations.

1.4 SCOPE

The project covers the full path of an incident inside the response system, from the moment it is reported to the moment its analysis is displayed to the analyst.

1.4.1 In Scope

- Incident collection through a web reporting form and REST API.
- Validation of submitted attributes and conversion into facts.
- Rule-based classification with forward-chaining inference.
- Heuristic priority scoring and severity levels (Critical, High, Medium, Low).

- Constraint-based response recommendation.
- Storage of incidents and analysis results in a MySQL database.
- Monitoring through a dashboard, incident list and reports page with CSV export.
- User sign-in and account creation.

1.4.2 Out of Scope

- Raw packet capture and network intrusion detection; the system works on incidents that an analyst or an external tool has already identified.
- Automatic execution of responses on firewalls or network devices; responses are recommended to the analyst.
- Training of machine-learning models on labelled traffic datasets.

1.5 SECURITY REQUIREMENTS

Because the system stores sensitive information about attacks, targets and source addresses, it is itself a target. Table 1.2 lists the security requirements and the mechanism used for each.

Table 1.2 Security requirements and the mechanism used for each

Requirement	Mechanism in the system
Secure incident data management	Incident records created and changed only through validated backend service methods
Controlled access to incident information	Username and password sign-in before any page is shown
Secure database integration	Database reached only through the repository layer; no direct client access
Reliable incident classification	Fixed, reviewable rule base with forward-chaining inference
Prevention of unauthorized modification	Changes made only through authenticated API calls
Secure communication between components	Defined REST/JSON interface between front end and backend
Accurate prioritization of critical incidents	Heuristic score with keyword escalation for admin, database and root targets

1.6 SDG ALIGNMENT

The project aligns with **Sustainable Development Goal 9: Industry, Innovation and Infrastructure**. It applies artificial intelligence to protect digital infrastructure by shortening the path from a raw incident to a prioritized, explained response.

1.6.1 SDG 9 Targets

Table 1.3 SDG 9 targets addressed by the project

SDG 9 target	Project contribution
Target 9.1: Develop reliable, sustainable and resilient infrastructure	Faster and consistent triage keeps digital services available during brute-force, DoS and malware incidents
Target 9.5: Strengthen scientific research and technological capability	Applies AI reasoning methods (expert systems, heuristic search, constraint satisfaction) to a practical security operations problem

1.6.2 Project Contribution

The system helps organizations move from manual incident handling to intelligent, prioritized and AI-assisted incident response. A small security team can use it to see every incident in one place, identify the critical ones at the time of reporting and act on recommended responses, which supports technological capability and the resilience of digital infrastructure.

1.7 ORGANIZATION OF THE REPORT

- **Chapter 2** reviews existing incident handling approaches and the literature, identifies research gaps and introduces the proposed system.
- **Chapter 3** presents the system design: architecture, modules, data flow and UML diagrams, threat model, security controls and technologies.
- **Chapter 4** describes data collection, preprocessing, the AI algorithms, the implementation, and the results with user interface screenshots.
- **Chapter 5** concludes the report, outlines future work and lists the references and appendices.

CHAPTER 2
LITERATURE REVIEW

This chapter reviews how intrusion detection and incident analysis have been approached in published work and in practice. Fourteen sources were studied, covering detection models, detection systems, evaluation datasets, surveys and threat frameworks. Table 2.1 summarizes them in chronological order.

2.1 Literature Survey

Early work on intrusion detection laid the conceptual foundation for the field. Denning (1987) [8] proposed an intrusion-detection model based on statistical profiles of normal subject behaviour, where deviations from the profile are flagged as anomalies; however, these profiles must be tuned, and changes in legitimate behaviour lead to false alarms. Roesch (1999) [9] introduced Snort, a lightweight network intrusion detection system that uses rule-based signature matching on network packets, but it can only detect attacks for which a signature has been written. In the same year, Paxson (1999) [10] presented Bro, a system for detecting network intruders in real time through an event engine and a policy script interpreter. Its limitations are that it requires expert policy scripting and its output stops at detection.

Subsequent studies examined how intrusion detection systems should be evaluated and how reliable they are. Axelsson (2000) [11] applied Bayesian analysis to detector false-alarm rates and showed that false alarms dominate in practice (the base-rate fallacy), although the work offers no method for triaging alerts. Lippmann et al. (2000) [12] carried out the 1998 DARPA off-line evaluation of intrusion detection systems using generated background and attack traffic, but the traffic was synthetic and the attacks have since become dated.

As machine learning entered the domain, researchers reviewed anomaly-based approaches and their practical difficulties. Garcia-Teodoro et al. (2009) [13] reviewed statistical, knowledge-based and machine learning techniques for anomaly-based network intrusion detection, noting high false-positive rates and the difficulty of evaluation. Tavallaee et al. (2009) [15] performed a detailed statistical analysis of the KDD CUP 99 data set and created NSL-KDD, yet the dataset remains dated for modern attacks. Sommer and Paxson (2010) [14] analysed why machine learning-based anomaly detection struggles in operational

networks, citing the high cost of errors, the diversity of traffic and the semantic gap between detection and meaningful interpretation.

Other works approached the problem from the viewpoint of attacker behaviour. Hutchins et al. (2011) [3] proposed an intelligence-driven defence model based on the intrusion kill chain, which describes the phases of an attack, but the model is conceptual and provides no automated triage. Strom et al. (2018) [4] described MITRE ATT&CK, a knowledge base of adversary tactics and techniques; it describes behaviour but does not prioritize incidents.

Comprehensive surveys and newer datasets further shaped the field. Liao et al. (2013) [16] classified intrusion detection systems by method, approach and technology, though their survey is limited to detection. Buczak and Guven (2016) [17] surveyed data mining and machine learning methods for misuse and anomaly detection, finding that performance depends on the data, features and attack context. Sharafaldin et al. (2018) [18] introduced the CICIDS2017 dataset with realistic benign profiles, but it is laboratory-generated and serves detection rather than response. Finally, Khraisat et al. (2019) [19] reviewed signature-based and anomaly-based systems along with available datasets and concluded that no single technique suits every attack.

Overall, the existing literature concentrates heavily on detecting intrusions, whether through signatures, anomaly profiles or machine learning, while giving limited attention to prioritizing and responding to the resulting incidents. This gap motivates the present work.

2.1.1 EXISTING SYSTEM WITH SECURITY APPROACHES

In most organizations incidents are handled by a combination of manual investigation, rule or signature-based intrusion detection, statistical or machine-learning anomaly detection and separate monitoring and ticketing tools. Each approach is described below.

2.1.2 Manual Incident Investigation

In the manual approach an analyst reads each alert, gathers context from logs and other tools, decides whether it is a real incident, estimates its severity and chooses a response. The process follows the detection and analysis phase described by NIST [1]. It works for small volumes but does not scale, and its outcome depends heavily on the experience of the analyst on duty.

2.1.3 Signature-Based Intrusion Detection

Signature-based systems such as Snort [9] compare traffic with rules that describe known attacks, and Bro [10] adds a scripting language for site-specific policies. NIST SP 800-94 [2] describes such systems as the most common form of intrusion detection and prevention. Their alerts are easy to interpret because each one is tied to a rule, but they cannot recognize attacks for which no rule exists, and they report detections without ranking them.

2.1.4 Anomaly and Machine Learning Based Detection

Anomaly detection, introduced by Denning [8], models normal behaviour and flags deviations. Later work applied statistical, knowledge-based and machine-learning techniques [13], [16], [17], [19], evaluated on datasets such as DARPA 1998 [12], KDD'99 and NSL-KDD [15] and CICIDS2017 [18]. Sommer and Paxson [14] explain why such detectors are hard to deploy: errors are costly, traffic varies widely between networks, and an anomaly score does not tell an analyst what happened. Axelsson [11] adds that the low base rate of attacks makes false alarms unavoidable at scale.

2.1.5 Security Monitoring and Separate Response Tools

Alerts from detection tools are usually collected in a monitoring console or ticketing system, while response procedures are kept in documents or in the memory of experienced staff. Threat frameworks such as the kill chain [3] and ATT&CK [4] help analysts understand attacker behaviour but do not by themselves decide which incident to handle first.

2.1.6 Existing System Workflow

Figure 2.1 shows the typical workflow followed in the existing system and the limitation introduced at each step.

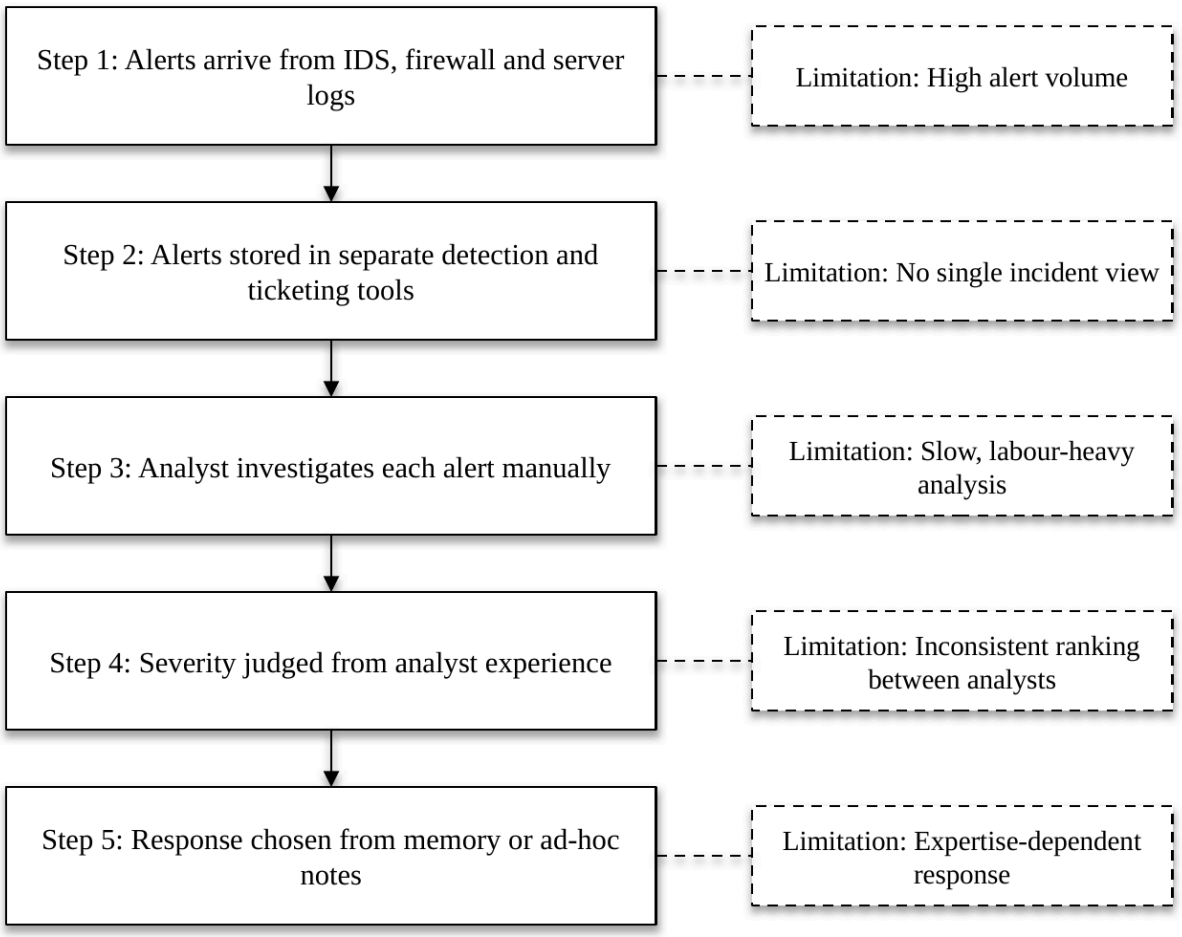


Figure 2.1 Existing incident handling workflow (manual triage process) with its limitations

2.1.7 Advantages and Limitations

Table 2.2 Advantages and limitations of the existing system

Advantages	Limitations
Simple to implement	Large numbers of alerts overwhelm security teams
Detects known attack patterns	Manual analysis is time-consuming
Useful for basic security monitoring	Limited intelligent prioritization
Established security techniques are available	Response decisions depend heavily on human expertise
Detection rules are easy to read	Detection and response are handled by separate systems

2.2 RESEARCH GAPS

2.2.1 Gaps Identified

The surveyed work concentrates on detecting attacks. Much less attention is given to what happens after an incident is recorded: deciding how serious it is, which one to handle first and what to do about it. Table 2.3 maps each gap to the way this project responds.

Table 2.3 Research gaps and project response

Research gap	How this project responds
Machine learning is hard to apply reliably in operational networks [14]	Rule-based expert system with explicit, auditable rules
ML methods need labelled datasets and feature engineering [17], [19]	Rules work on incident attributes, so no training set is required
Real network data is hard to model accurately [14]	Reasoning over incident-level facts: type, source, target and failed attempts
No single detection technique suits every attack type [19]	Per-attack rule sets feeding one shared priority score
False alarms dominate detector output [11]	Every incident receives a priority, so analysts handle the highest-risk ones first
Weak support for prioritization and response	Heuristic priority score plus constraint-based response recommendation

2.2.2 Refined Research Gap

There is no lightweight, explainable layer that takes a recorded incident and returns its class, a priority and a valid response together with the rules that justify them. Filling this gap does not require a new detector; it requires a reasoning layer that sits after detection and before response.

2.3 PROPOSED SYSTEM

The proposed system is an **AI-powered web application** that records cybersecurity incidents, analyzes them with knowledge-based artificial intelligence and recommends response actions to the analyst.

2.3.1 Overview of the Proposed System

Each incident follows the workflow in Figure 2.2. Incident data is analyzed by the AI engine, classified, assigned a priority and answered with a response recommendation. The system

combines centralized incident management, rule-based reasoning, forward chaining, heuristic prioritization, constraint-based response selection and dashboard monitoring.

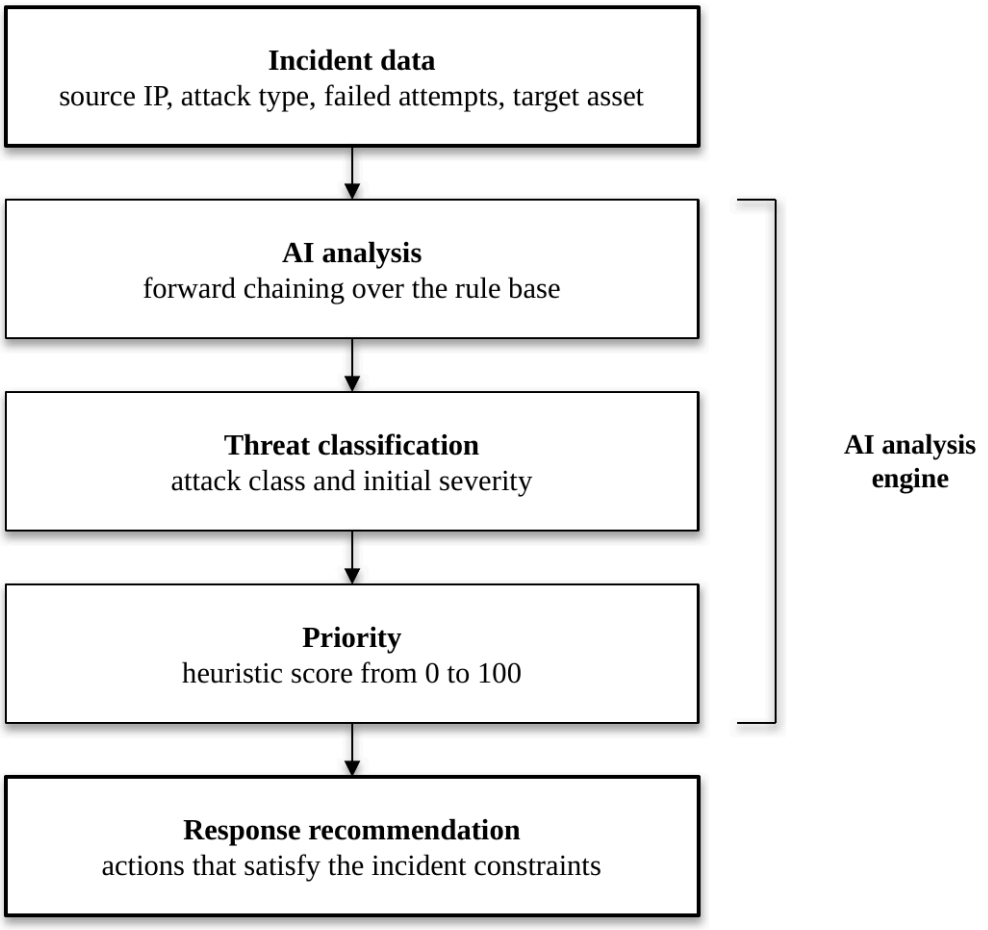


Figure 2.2 Proposed workflow of the AI-Powered Incident Response System

2.3.2 Objectives of the Proposed System

- Automate incident analysis.
- Classify security incidents into six attack classes.
- Determine incident severity (Critical, High, Medium, Low).
- Prioritize threats with a score from 0 to 100.
- Recommend appropriate responses.
- Provide centralized incident monitoring and reporting.
- Assist security teams in decision-making with explainable output.

2.3.3 Advantages of the Proposed System

- **Less manual analysis:** classification and scoring run as soon as an incident is submitted.
- **Consistent prioritization:** the same incident facts always produce the same severity and score.
- **Explainable decisions:** forward chaining records which rules fired for each conclusion.
- **Centralized information:** every incident, its analysis and its status are held in one MySQL database.
- **Faster response decisions:** recommended actions are shown next to each incident.
- **Integrated reporting:** dashboard, analytics and CSV export are part of the same application.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system is organized in four layers, shown in Figure 3.1. Each layer communicates only with the layer next to it, which keeps the AI logic separate from the user interface and from storage.

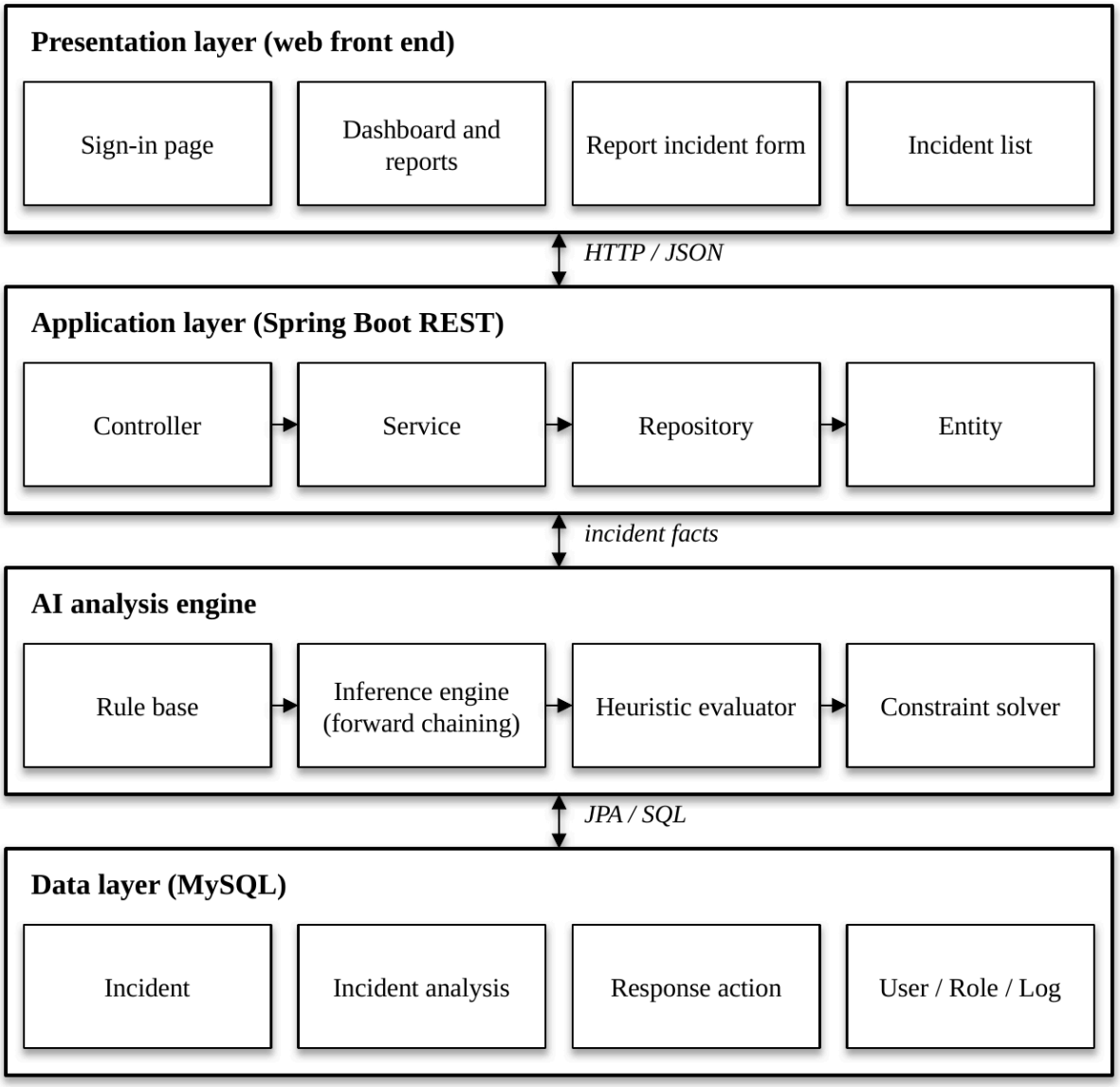


Figure 3.1 Layered system architecture of the AI-Powered Incident Response System

3.1.1 Presentation Layer

The presentation layer is the web front end used by the security analyst. It provides the sign-in page, the dashboard with summary cards and charts, the Report New Incident form,

the searchable incident list and the reports page. It communicates with the backend only through HTTP requests carrying JSON data.

3.1.2 Application Layer

The application layer is a Spring Boot service that follows the representational state transfer (REST) style [24]. The controller receives HTTP requests and returns responses, the service holds the business logic and calls the AI engine, the repository performs data access, and the entity classes map Java objects to database tables.

3.1.3 AI Analysis Engine

The AI analysis engine contains four components: the rule base that stores IF-THEN rules for each attack class, the inference engine that applies forward chaining, the heuristic evaluator that computes the priority score, and the constraint solver that selects valid response actions. The engine receives incident facts from the service layer and returns the classification, severity, priority and recommendation.

3.1.4 Data Layer

The data layer is a MySQL database. Its schema follows the class diagram in Section 3.3.5 and stores incidents, analysis results, response actions, users, roles and system logs. All access goes through the repository layer, so the front end never connects to the database directly.

3.2 MODULE DESIGN

The project is divided into four modules. Table 3.1 summarizes each module and its current status.

Table 3.1 System modules

Module	Function	Techniques	Status
M1 Incident Management and Backend	Collect, store, update and delete incident records	Spring Boot layers, MySQL, CRUD REST APIs	Completed
M2 AI-Based Incident Analysis	Classify incidents and establish the initial threat level	Rule base, forward chaining	Implemented
M3 Intelligent Prioritization and Response	Score priority and select response actions	Heuristic evaluation, constraint satisfaction	Implemented

Module	Function	Techniques	Status
M4 Dashboard and Integration	Monitoring, incident list, reporting and analytics	Web front end over REST APIs	Implemented

3.2.1 Module 1: Incident Management and Backend

Module 1 handles incident collection, storage and management. The Incident entity is mapped to a MySQL table through a repository; a service class holds the business logic; a controller exposes create, read, update and delete operations as REST endpoints. All four operations were tested with Postman. This module is the foundation on which the other three are built.

3.2.2 Module 2: AI-Based Incident Analysis

Module 2 converts the submitted incident into facts and runs forward chaining over the rule base to establish the attack classification and initial severity. The rules that fire are recorded as a trace, so the analyst can see why a conclusion was reached. The module needs no training data because its knowledge is written as rules.

3.2.3 Module 3: Intelligent Prioritization and Response

Module 3 evaluates severity with a heuristic function that considers asset criticality, attack frequency and impact, and produces a priority score from 0 to 100. Targets containing high-value keywords such as admin, database and root raise the severity threshold. The response stage then selects actions that satisfy the incident's constraints.

3.2.4 Module 4: Dashboard and Integration

Module 4 provides the analyst interface: sign-in, a dashboard with summary cards and charts, an attack-origin globe, a searchable incident list, an incident reporting form and a reports page with 7-day, 30-day and all-time filters and CSV export. It integrates the other modules through the REST API.

3.3 DATA FLOW / UML DIAGRAMS

The design is described with two data flow diagrams and four UML diagrams. Together they show what data moves through the system, who uses it, in what order activities occur, how the data is structured and how components interact over time.

3.3.1 Data Flow Diagram Level 0

The Level 0 diagram in Figure 3.2 treats the whole system as one process. Incident information arrives from security logs and from the analyst, records move to and from the MySQL data store, processed data goes to the AI analysis engine, and classification, priority and response come back to the analyst as analysis results.

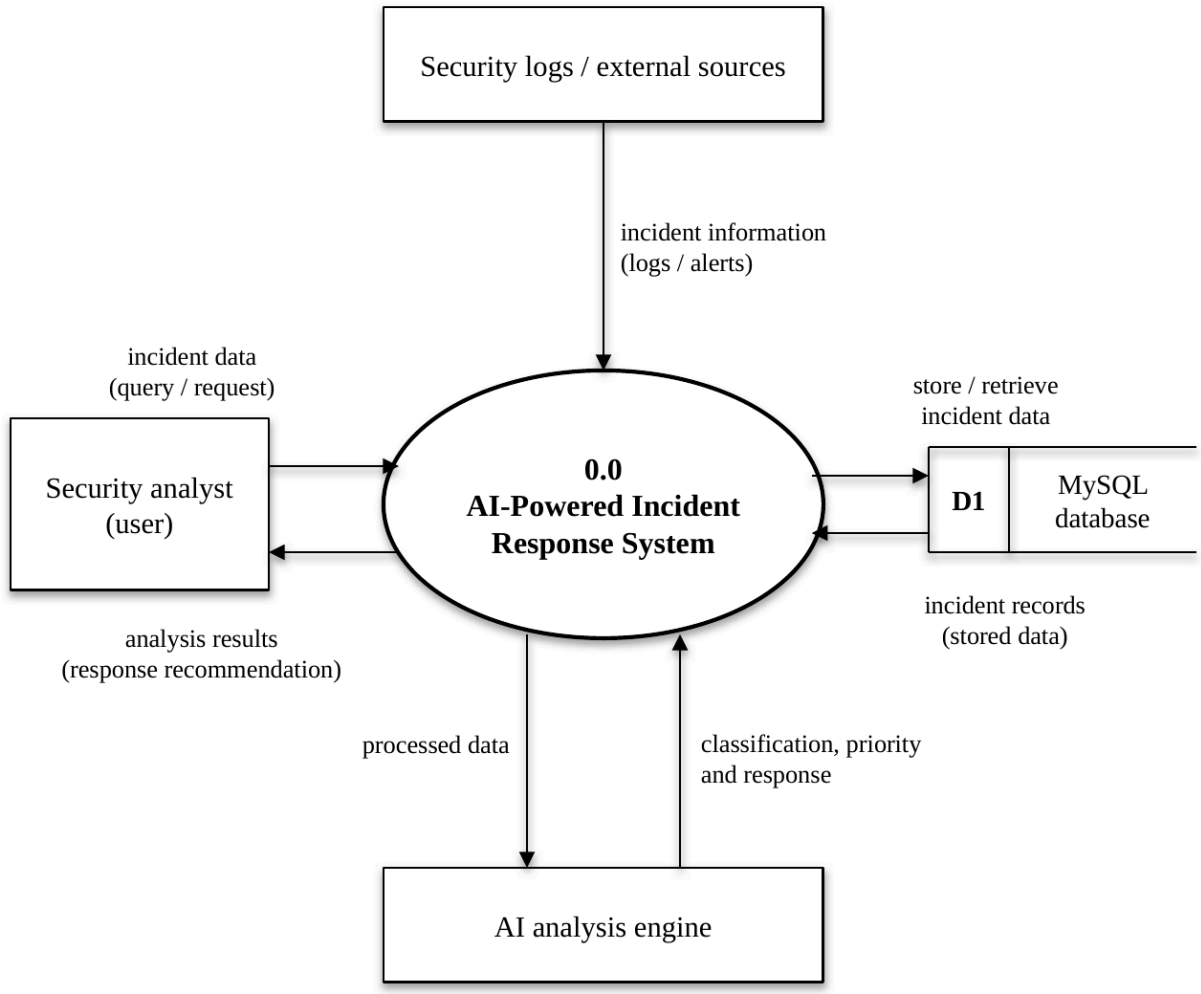
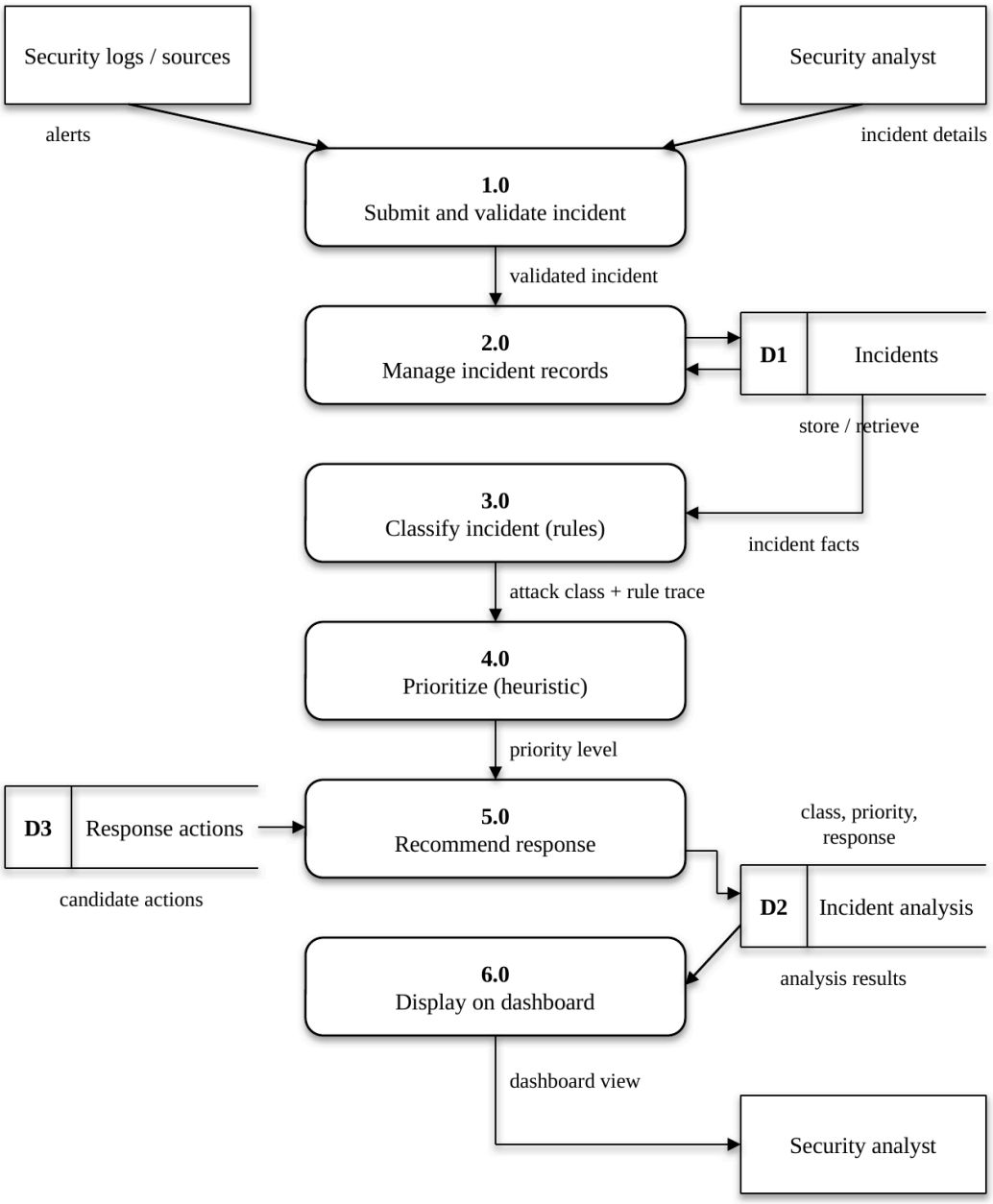


Figure 3.2 Level 0 data flow diagram of the AI-Powered Incident Response System

3.3.2 Data Flow Diagram Level 1

Figure 3.3 expands the single process into six sub-processes: submit and validate incident (1.0), manage incident records (2.0), classify incident (3.0), prioritize with the heuristic (4.0), recommend response (5.0) and display on the dashboard (6.0). The data stores are D1 Incidents, D2 Incident analysis and D3 Response actions. Processes 1.0 and 2.0 belong to Module 1, process 3.0 to Module 2, processes 4.0 and 5.0 to Module 3, and process 6.0 to Module 4.



Rectangle: external entity Rounded box: process Open box: data store Arrow: data flow

Figure 3.3 Level 1 data flow diagram showing Modules 1 to 4 of the AI-Powered Incident Response System

3.3.3 Use Case Diagram

The use case diagram in Figure 3.4 has two actors. The security analyst manages incidents, views incident analysis, views priority and response recommendations, searches and filters incidents and opens dashboard reports. The administrator manages users and system settings. Viewing analysis and recommendations includes the use cases of the AI analysis module: classify incident, determine priority and recommend response.

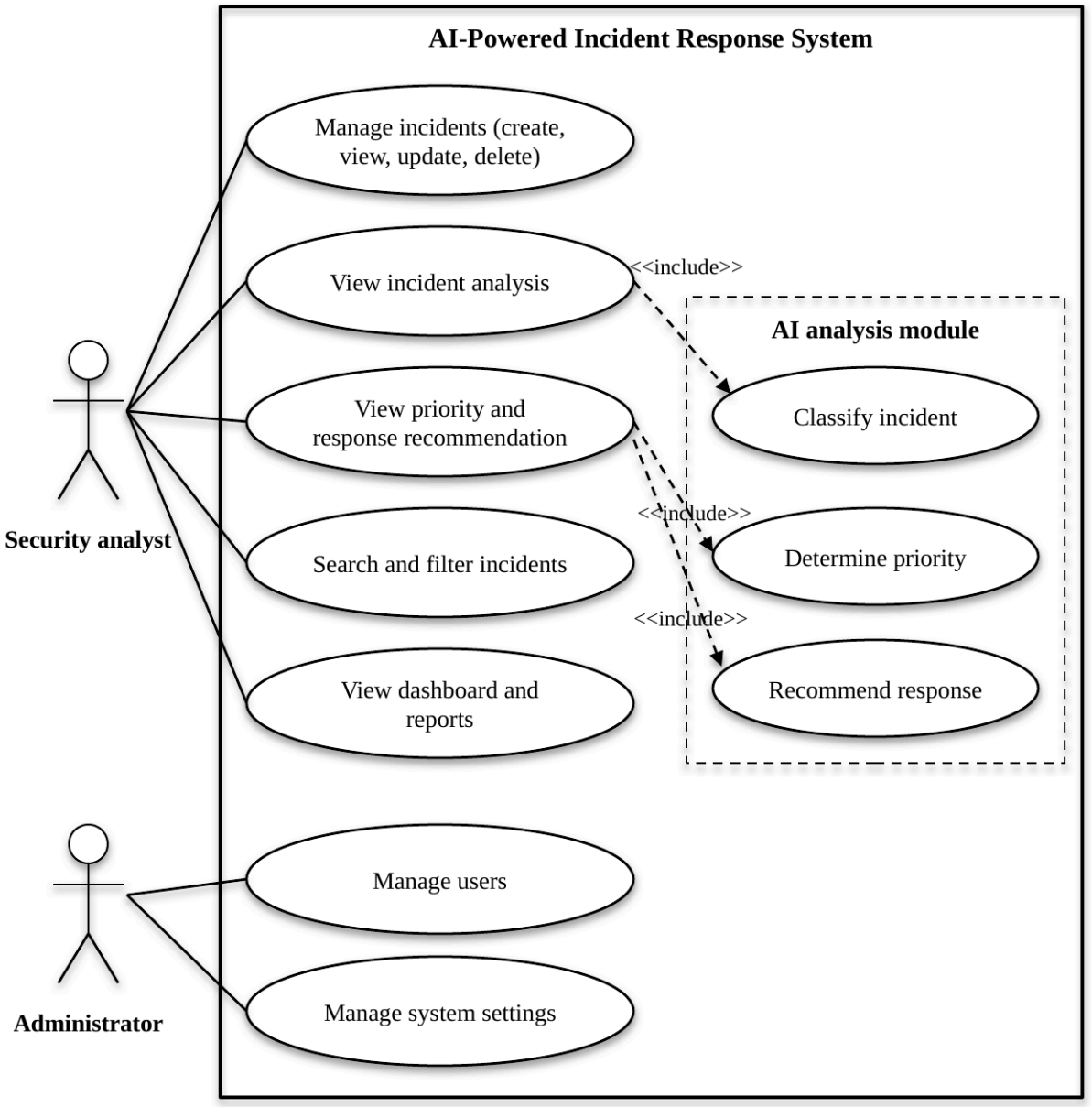


Figure 3.4 Use case diagram of the AI-Powered Incident Response System showing the analyst, administrator and AI analysis module

3.3.4 Activity Diagram

The activity diagram in Figure 3.5 follows one incident from submission to display. Invalid input returns a validation error to the analyst. Valid incidents are stored, preprocessed, classified, scored and given a recommendation; Critical incidents are also added to the Needs Attention list on the dashboard.

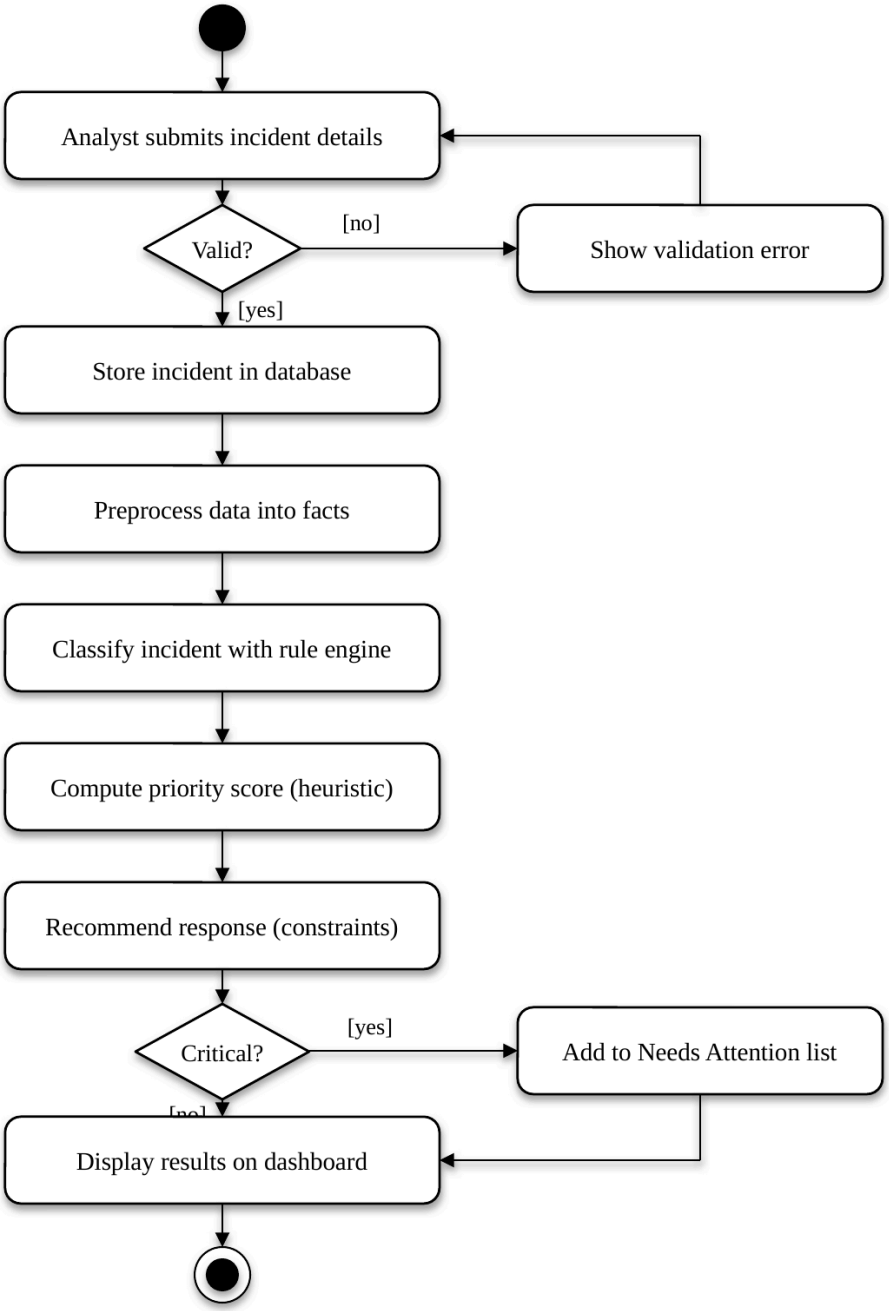


Figure 3.5 Activity diagram of incident submission and analysis across Modules 1 to 4

3.3.5 Class Diagram

The class diagram in Figure 3.6 contains six classes: User, Role, Incident, IncidentAnalysis, ResponseAction and SystemLog. One user creates many incidents, each incident has one analysis, and each analysis recommends one or more response actions. Each user has one role, and system logs record user actions for auditing.

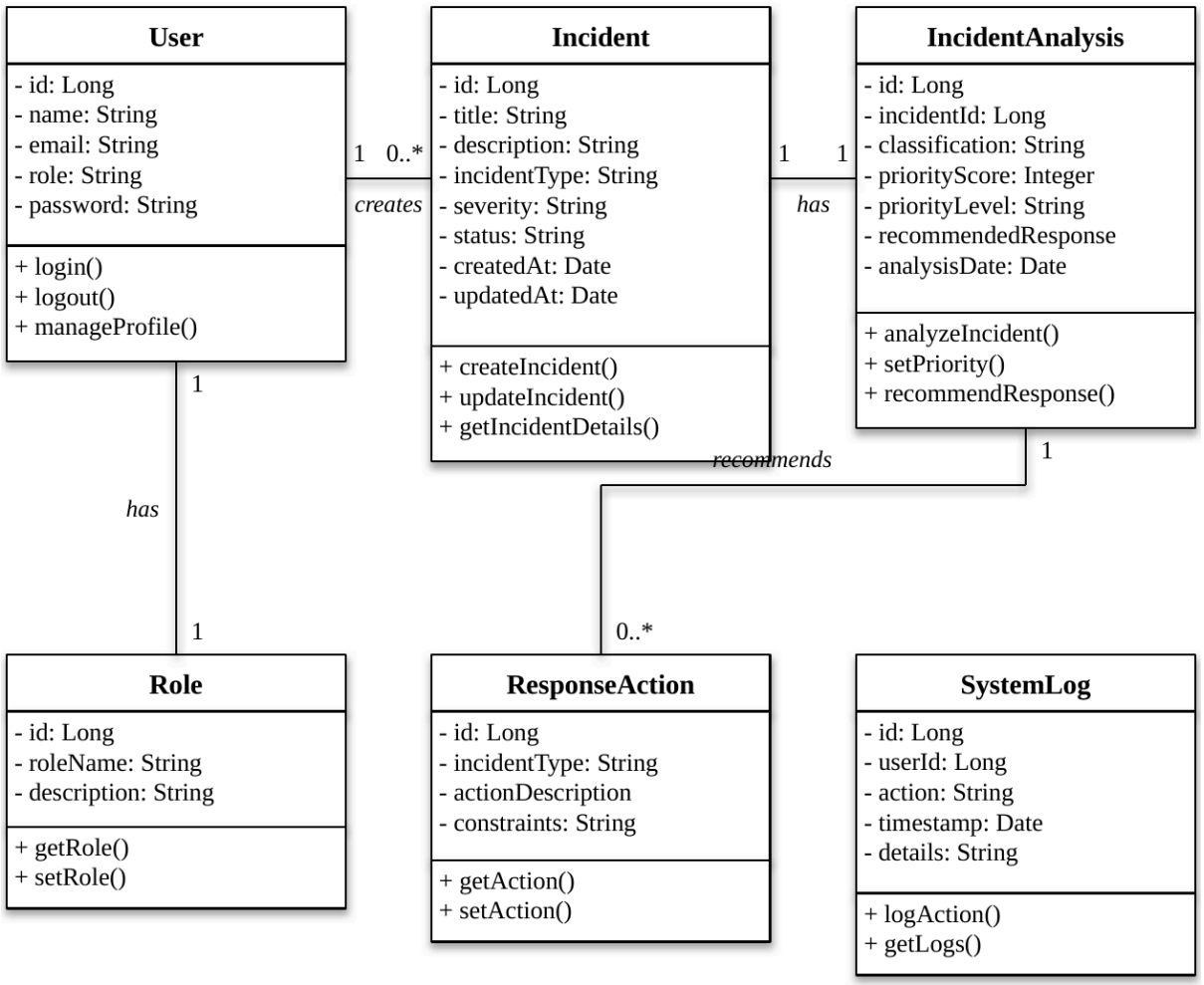


Figure 3.6 Class diagram of the incident data model (Module 1: Incident Management and Backend)

3.3.6 Sequence Diagram

The sequence diagram in Figure 3.7 shows the ten messages exchanged when an analyst submits an incident. The controller passes the incident to the service, which stores it in MySQL. The controller then calls the AI analysis engine for classification, priority and response, saves the results and returns them to the dashboard.

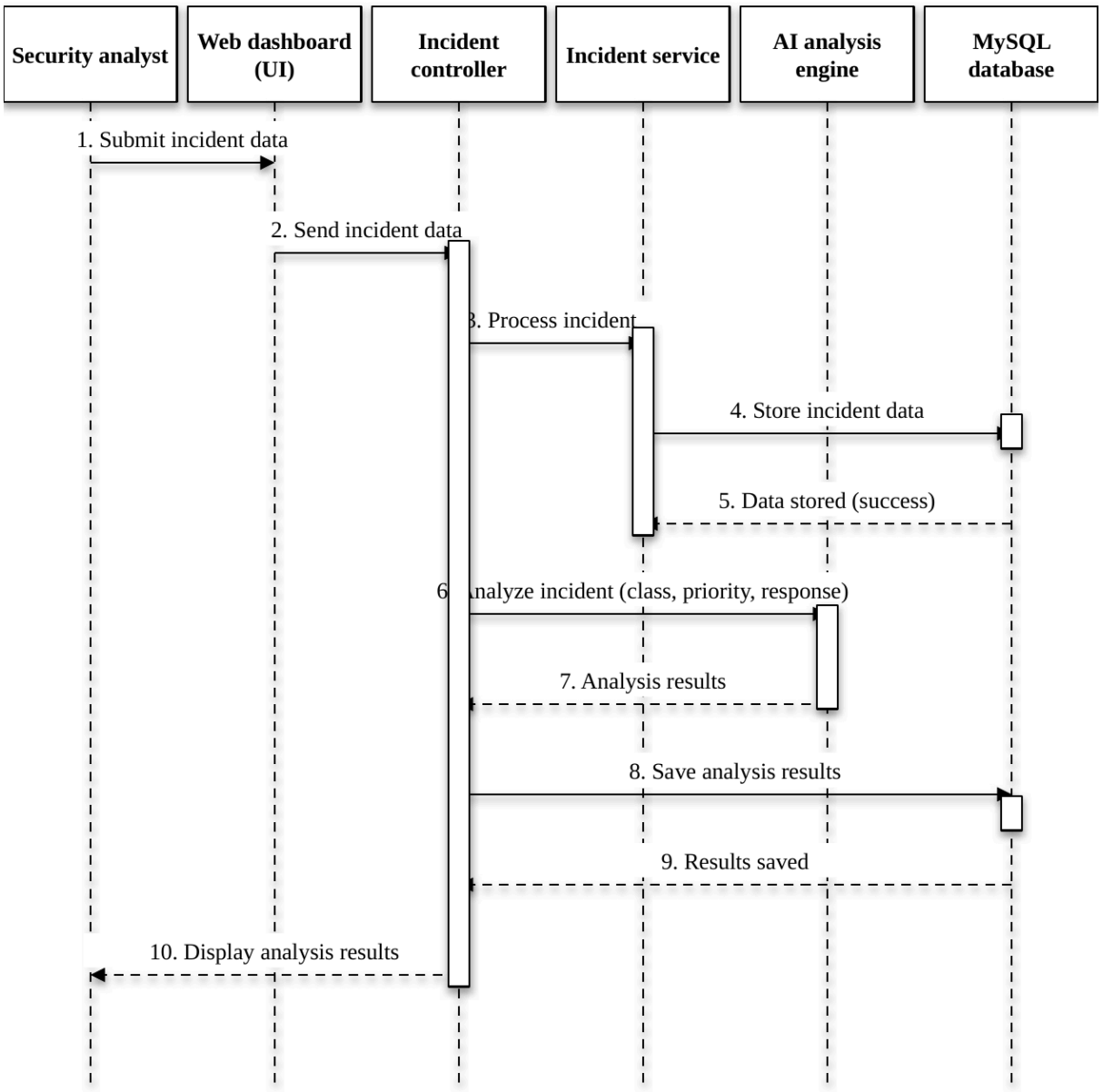
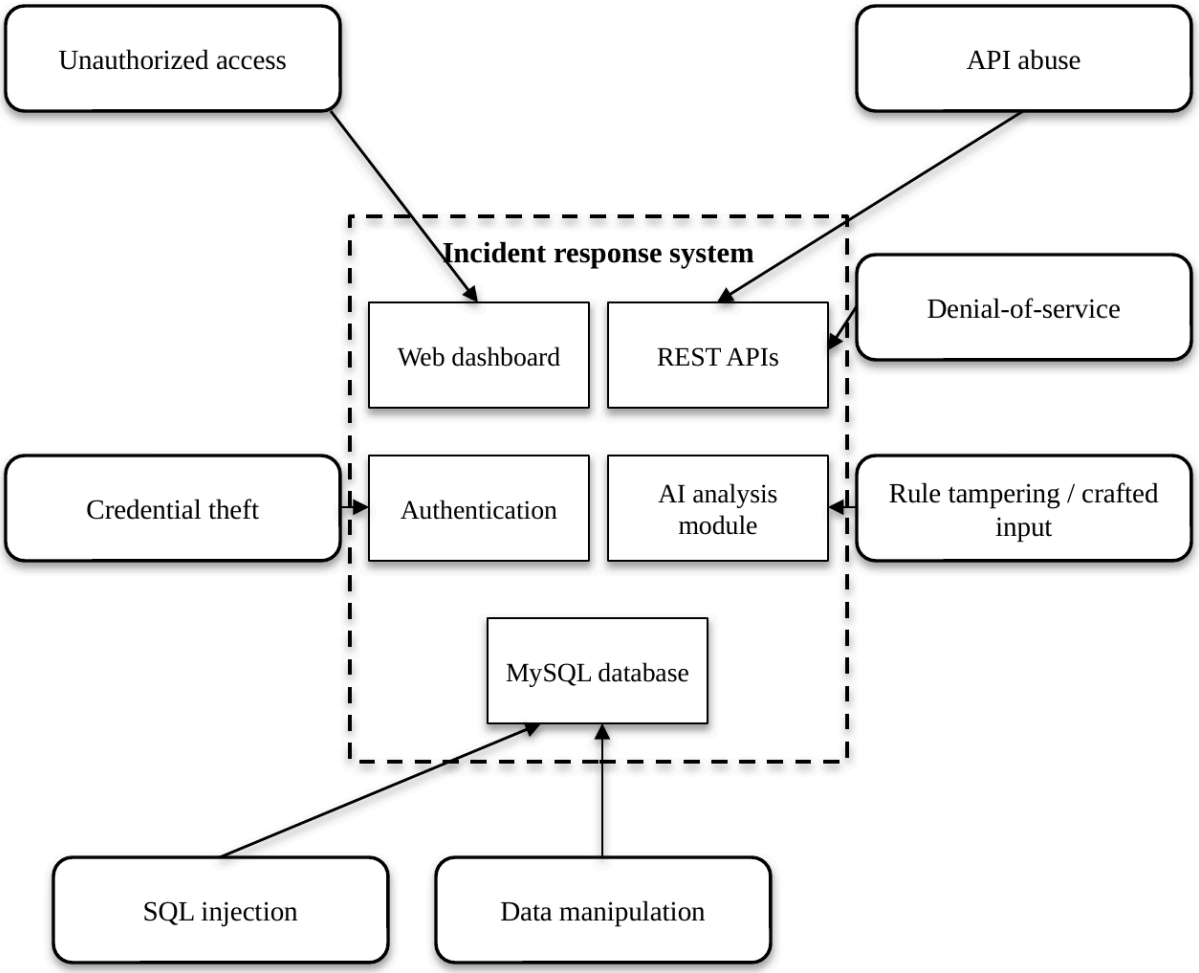


Figure 3.7 Sequence diagram of the incident analysis flow (Modules 1, 2 and 3)

3.4 THREAT MODEL AND ATTACK SURFACE

3.4.1 Attack Surfaces

The system handles sensitive incident data through the web dashboard, REST APIs, authentication system, AI analysis module and MySQL database. These five components form the attack surface shown in Figure 3.8, and each is exposed to one or more of the threats listed in the OWASP Top 10 [25].



Rounded box: threat Rectangle: attack surface Arrow: threat targets surface

Figure 3.8 Threat model and attack surfaces of the AI-Powered Incident Response System

3.4.2 Threat Analysis

Table 3.2 Threat model and security controls

Threat	Attack surface	Control
Unauthorized access	Dashboard, authentication	Sign-in with username and password; role-based authorization (User, Role)
SQL injection	MySQL via REST APIs	Input validation and parameterized database queries
API abuse	REST APIs	Rejection of unauthenticated requests; request controls
Data manipulation	Incident records	Integrity checks; user actions recorded in SystemLog
Credential theft	Authentication system	Restricted access, logging and monitoring of user actions

Threat	Attack surface	Control
Denial-of-service	APIs, dashboard	Error handling, backups and request controls
Rule tampering or crafted input	AI analysis module	Controlled rule set and validated inputs

3.5 SECURITY CONTROLS

- **Authentication and authorization:** only signed-in users reach the application, and roles restrict administrative actions.
- **Input validation:** required fields, IPv4 format and numeric attempt counts are checked before analysis.
- **API security:** REST endpoints accept only authorized requests.
- **Database security:** access goes through the repository layer with parameterized queries to prevent SQL injection.
- **AI security:** the rule base is controlled and every input is validated before inference.
- **Logging and monitoring:** user and system actions are recorded for audit.
- **Data integrity:** incident records cannot be modified outside authorized operations.
- **Availability:** error handling, backups and request controls keep the service available.

3.6 TOOLS AND TECHNOLOGIES

Table 3.3 Technologies used

Layer	Technology	Purpose and justification
Language	Java	Typed, mature ecosystem; one language for backend services and the rule engine
Backend framework	Spring Boot (REST)	Layered entity, repository, service and controller structure with little configuration
Database	MySQL	Relational schema fits structured incident and analysis records
API testing	Postman	Repeatable checks of every CRUD endpoint
Front end	Web single-page application	Sign-in, dashboard, incident forms and reports in the browser
Visualization	Charts and 3D globe in the front end	Trend, distribution and attack-origin views for analysts

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

Incidents enter the system through the Report New Incident form or the REST API. The system works on incident-level data rather than raw packets, so no external training dataset is required.

4.1.1 Incident Input Attributes

The analyst records the attributes in Table 4.1. The system adds an identifier, the reporting time and an initial status of Open.

Table 4.1 Incident input attributes

Attribute	Type	Required	Use in analysis
Source IP address	IPv4 text, for example 192.168.1.105	No	Repeat-source detection; geolocation on the attack-origin globe
Attack classification	One of six classes	Yes	Selects the rule set for forward chaining
Failed attempts	Integer, for example 15	No	Attack frequency factor in the heuristic score
Target asset	Text, for example Admin Gateway, DB-Prod-01, /api/auth	No	Asset criticality; keywords admin, database and root raise severity

4.1.2 Simulated and Recorded Incidents

A Simulate Attack control on the dashboard creates sample incidents so that the analysis pipeline and dashboard can be tested without live attacks. During testing, nine incidents were recorded between 3 and 9 October 2026 covering brute force, malware, DDoS and port scan. These incidents form the test data set used in Section 4.2.2.

4.2 DATA PREPROCESSING

Before analysis the submitted record is checked and turned into facts that the rule engine can use. The processing pipeline has six stages:

- **Incident data collection:** attributes are received from the form or API.

- **Data preprocessing:** required fields are checked, the IP format is verified, and text is trimmed and normalized.
- **Feature processing:** values are converted into facts such as attack type, attempt count and target keywords.
- **AI-based analysis:** forward chaining over the rule base derives the classification.
- **Heuristic evaluation:** severity and a priority score from 0 to 100 are computed.
- **Response recommendation:** actions that satisfy the incident's constraints are attached to the record.

An incident is represented as the tuple $x = (s, a, n, t)$, where s is the source IPv4 address, a is the attack class from the set A of six classes, n is the number of failed attempts and t is the target asset. Algorithm 4.1 shows how the input is validated and converted into the initial fact set F_0.

Algorithm 4.1 Input validation and fact generation

Input: $x = (s, a, n, t)$, attack classes A, keyword set $K_w = \{admin, database, root\}$

Output: initial fact set F_0 or a validation error

Step 1:if $a \notin A$ **then** reject the incident with a validation error

Step 2:if $s \neq \emptyset$ **and** s is not a valid IPv4 address **then** reject the incident

Step 3:if $n \neq \emptyset$ **and** $n < 0$ **then** reject the incident

Step 4:$K_t \leftarrow$ keywords of K_w that occur in t

Step 5:$F_0 \leftarrow \{attack(a),\ source(s),\ attempts(n),\ target(t)\} \cup \{keyword(k) :\ k \in K_t\}$

Step 6:return F_0

4.2.1 Algorithms / Methodology

Figure 4.1 shows the complete methodology. Steps 3 to 6, marked with a bracket, form the AI core of the system and are described in the following subsections.

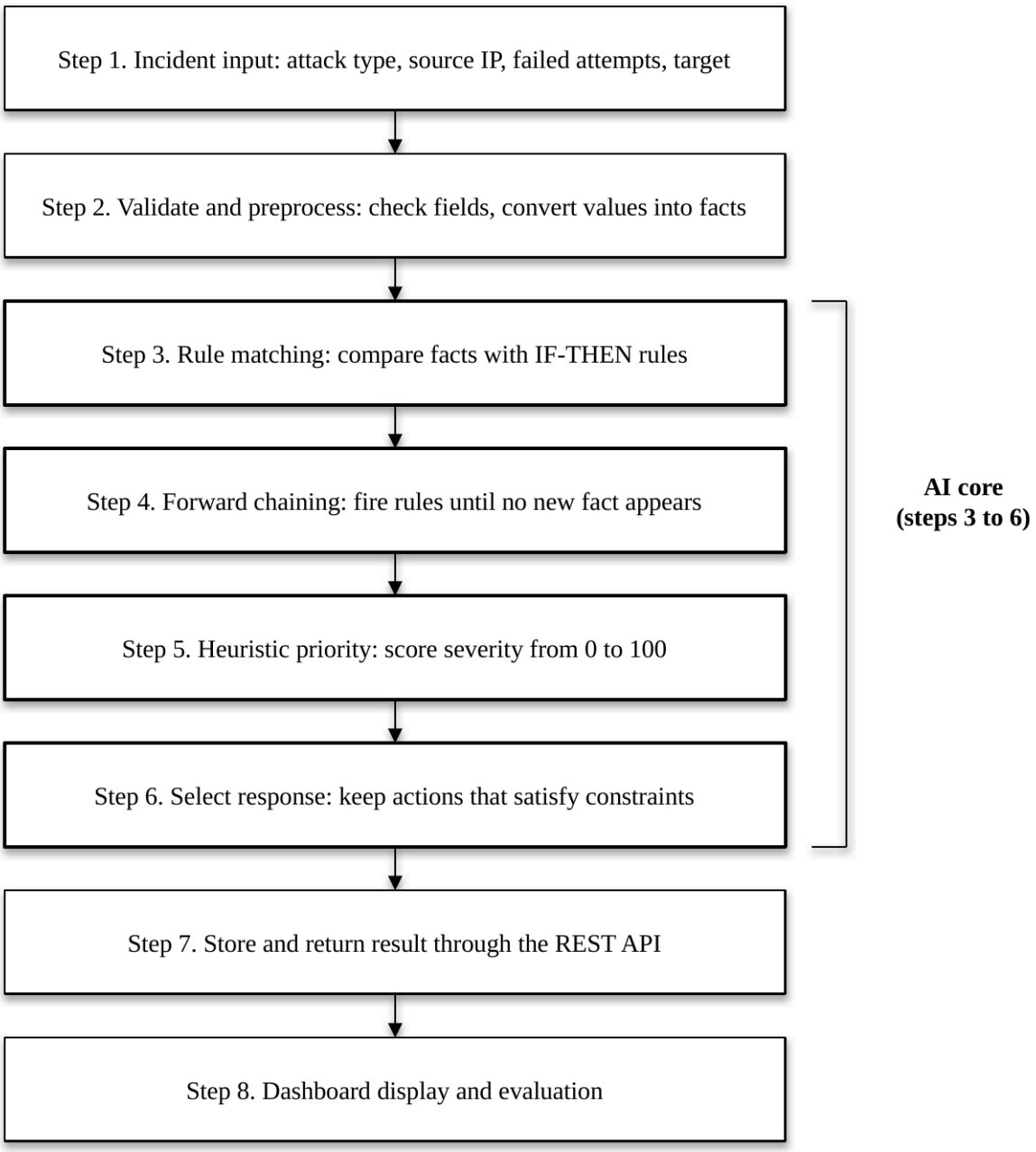


Figure 4.1 Methodology workflow of the AI analysis pipeline across Modules 1 to 4

4.2.1.1 Rule-Based Reasoning

Knowledge is stored as IF condition THEN conclusion rules written from security domain knowledge. Each rule r has a condition set C_r and a conclusion c_r. Rules are grouped by attack class, so a brute-force incident and a port scan are judged by different evidence. Table 4.2 shows the structure of representative rules.

Table 4.2 Representative rule structure

Rule	IF (conditions)	THEN (conclusion)
R1	attack = Brute Force AND failed attempts above threshold	brute-force attack confirmed
R2	brute-force confirmed AND target contains admin or root	severity = Critical
R3	attack = Malware AND target contains database	high-value asset involved
R4	attack = DDoS AND target is a public service	availability impact
R5	severity is High or Critical	notify analyst immediately

4.2.1.2 Forward Chaining

Forward chaining is data-driven inference [5]. It starts from the known incident facts, fires every rule whose conditions hold, adds the conclusions as new facts and repeats until no new fact can be derived. Because the set of derivable facts is finite, the procedure always terminates. With simple matching each pass checks every rule against the fact set; the Rete algorithm [20] can reduce this cost for large rule bases. The list of fired rules becomes the explanation shown to the analyst. Algorithm 4.2 gives the procedure and Figure 4.2 shows it as a flowchart.

Algorithm 4.2 Forward chaining with rule trace

Input: initial facts F_0, rule base R

Output: derived fact set F and rule trace T

Step 1: $F \leftarrow F_0,\ T \leftarrow \langle\,\rangle$

Step 2: **repeat**

Step 3: $N \leftarrow \emptyset$

Step 4: **for each** rule $r \in R$ **do**

Step 5: **if** $C_r \subseteq F$ **and** $c_r \notin F$ **then** $N \leftarrow N \cup \{c_r\},\ T \leftarrow T \oplus r$

Step 6: $F \leftarrow F \cup N$

Step 7: **until** $N = \emptyset$

Step 8: **return** (F, T)

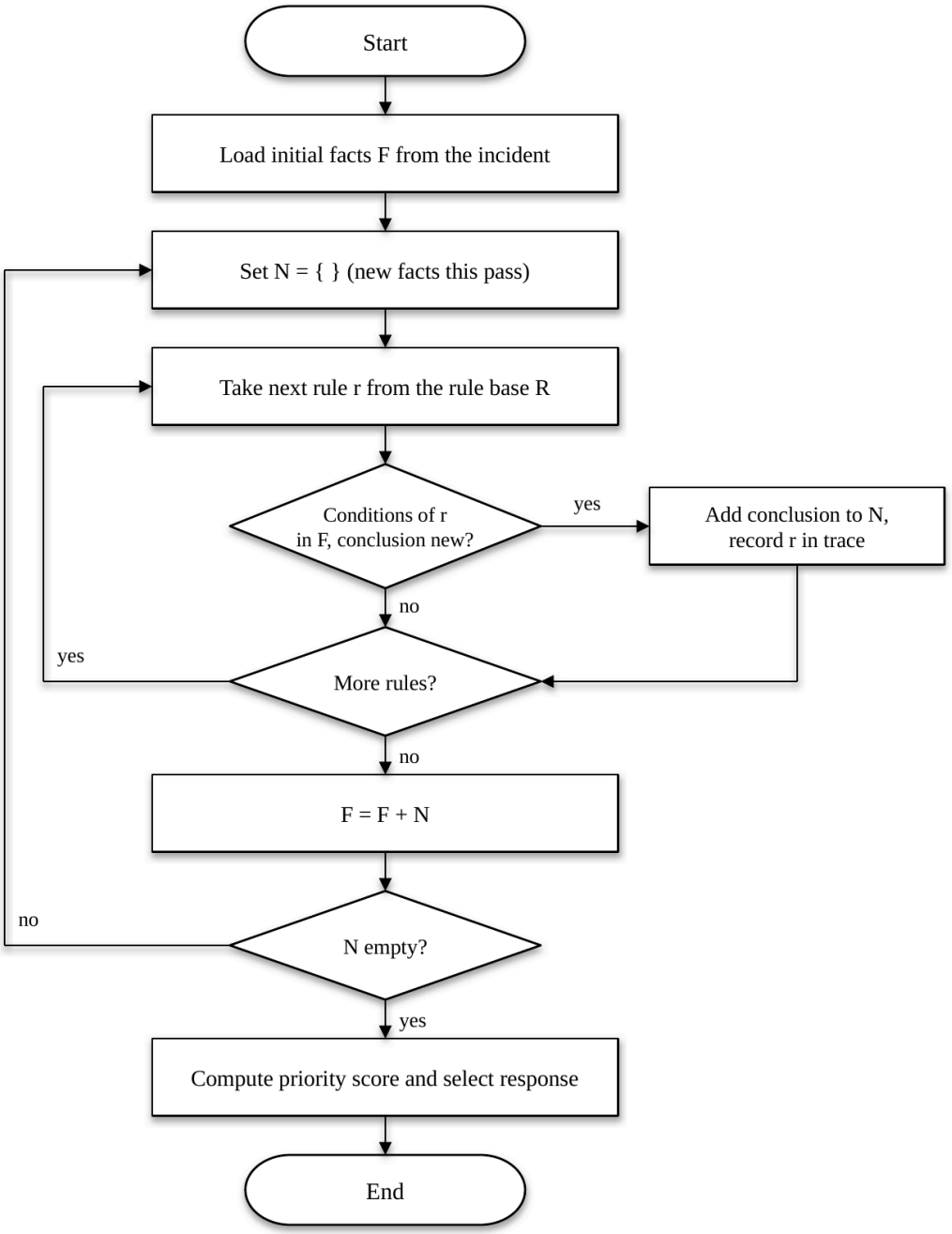


Figure 4.2 Forward-chaining inference flowchart (Module 2: AI-Based Incident Analysis)

4.2.1.3 Heuristic Evaluation

Instead of examining every possible outcome, the system scores an incident with a heuristic function of its characteristics, in the same spirit as the evaluation function used by A* search [22]. Three factors are used, each normalized to the range 0 to 1: asset criticality $C(t)$, derived from the target type and keywords; attack frequency $\Phi(n, s)$, derived from failed attempts and repeated sources; and attack impact $I(a)$, which is fixed per attack class. The

priority score is given by Equation (4.1), where the weights are non-negative and sum to one, so the score lies between 0 and 100.

$$P = 100 \cdot (w_1 C(t) + w_2 \Phi(n, s) + w_3 I(a)) \quad (4.1)$$

$$w_1 + w_2 + w_3 = 1, \quad \Phi(n, s) = min(1, \frac{n}{n_{max}}) \quad (4.2)$$

The score is mapped to a severity level using thresholds $\theta_1 > \theta_2 > \theta_3$. A keyword floor raises incidents on admin, database or root targets, and rule R2 can set the severity to Critical directly. The Common Vulnerability Scoring System [21] uses a similar idea, computing a fixed formula over a few metrics to give consistent severity scores. Algorithm 4.3 and Figure 4.3 describe the scoring.

Algorithm 4.3 Heuristic priority scoring

Input: derived facts F, weights w_1, w_2, w_3, thresholds $\theta_1, \theta_2, \theta_3$

Output: priority score P and severity level S

Step 1: $C \leftarrow C(t), \ \Phi \leftarrow min(1, \frac{n}{n_{max}}), \ I \leftarrow I(a)$

Step 2: $P \leftarrow 100 \cdot (w_1 C + w_2 \Phi + w_3 I)$

Step 3: **if** $severity(Critical) \in F$ **or** $P \geq \theta_1$ **then** $S \leftarrow Critical$

Step 4: **else if** $P \geq \theta_2$ **then** $S \leftarrow High$

Step 5: **else if** $P \geq \theta_3$ **then** $S \leftarrow Medium$ **else** $S \leftarrow Low$

Step 6: **return** (P, S)

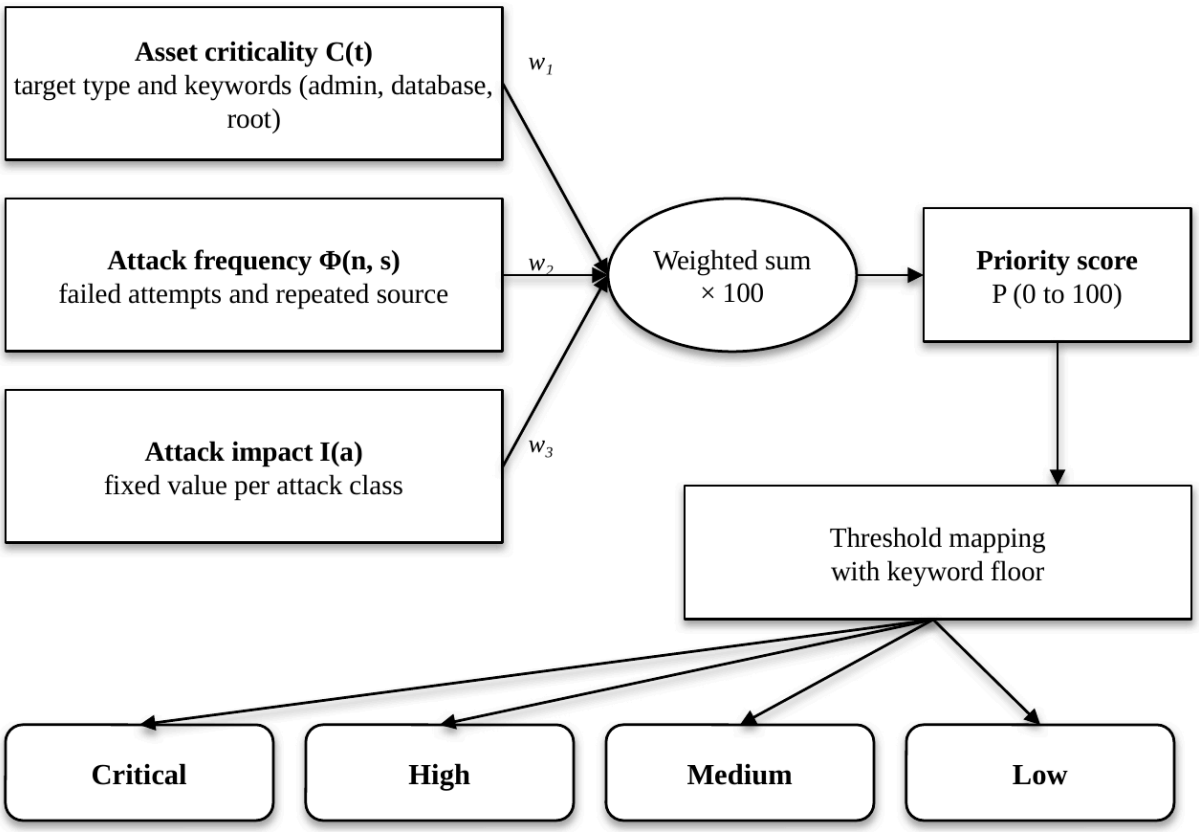


Figure 4.3 Heuristic priority scoring model (Module 3: Intelligent Prioritization and Response)

4.2.1.4 Constraint Satisfaction

Response selection is treated as a constraint satisfaction problem [5], [23]. For the derived attack class c the system holds a set of candidate actions A_c, such as blocking a source address, locking and resetting an account, rate-limiting a service or isolating a host. Constraints K encode conditions such as never auto-blocking a whitelisted internal host and requiring analyst escalation for Critical incidents. Only actions that satisfy every constraint are recommended, as given in Equation (4.3), Algorithm 4.4 and Figure 4.4.

$$R^{*} = \{\alpha \in A_c : k(\alpha, F) = true, \forall k \in K\} \quad (4.3)$$

Algorithm 4.4 Constraint-based response selection

Input: attack class c, facts F, severity S, constraints K

Output: recommended action set R^*

Step 1: $A_c \leftarrow$ candidate actions for class c

Step 2: $R^* \leftarrow \{\alpha \in A_c : \forall k \in K,\ k(\alpha, F) = true\}$

Step 3: **if** $S = Critical$ **then** add the action escalate to analyst to R^*

Step 4: **return** R^*

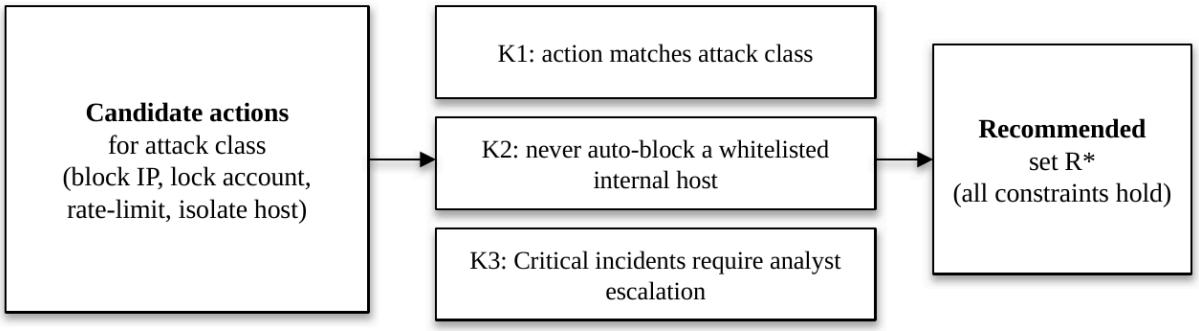


Figure 4.4 Constraint-based response selection (Module 3: Intelligent Prioritization and Response)

4.2.1.5 Worked Example

Figure 4.5 traces a brute-force incident from 10.0.0.5 against an administrator account through the pipeline. Rule R1 confirms the brute-force attack, rule R2 sets the severity to Critical because the target contains admin, the heuristic raises the priority, the constraints confirm that the source is not whitelisted and that escalation is required, and the system recommends blocking the source, locking and resetting the account and alerting the analyst.

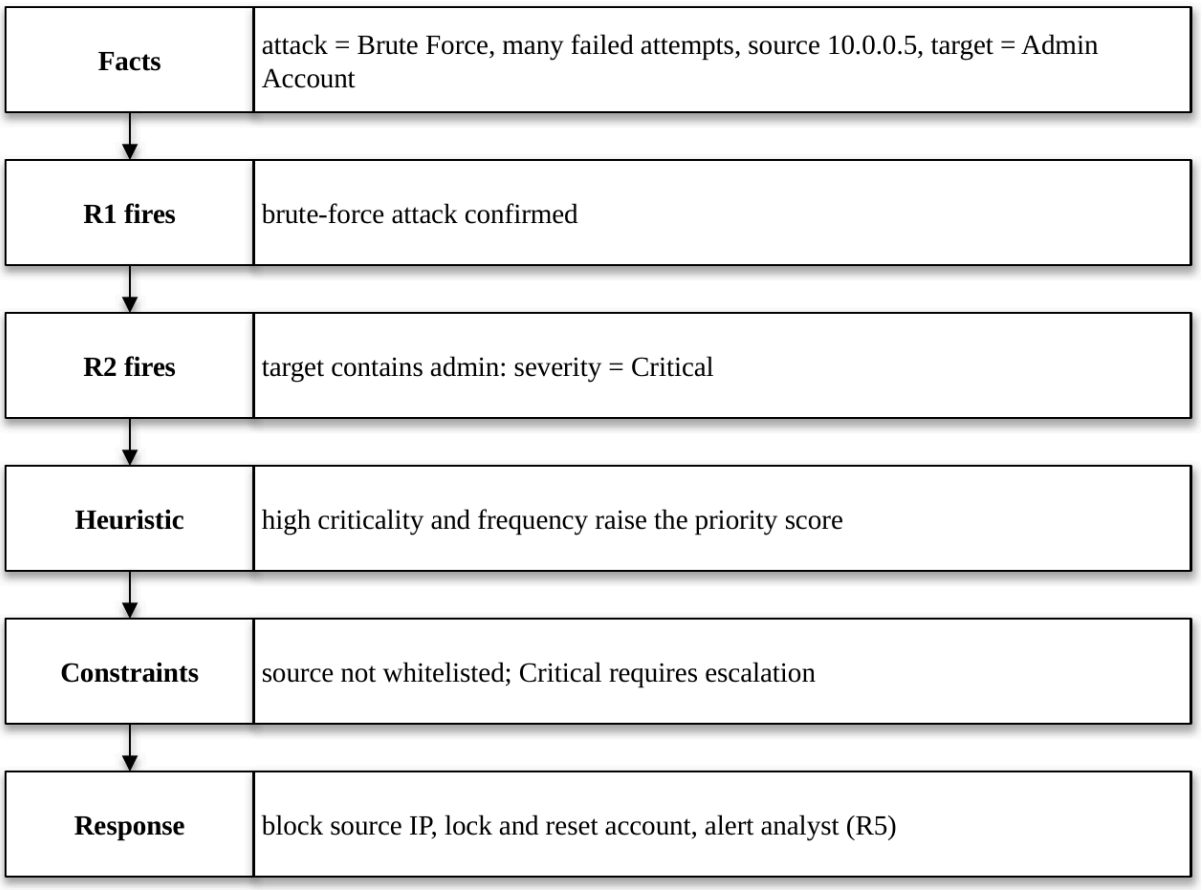


Figure 4.5 Inference trace for a brute-force incident (Modules 2 and 3)

4.2.2 Results and Discussion

4.2.2.1 Implementation Details

The backend is written in Java with Spring Boot. The Incident entity is persisted through a repository, a service class holds the business logic and calls the analysis pipeline, and a controller exposes create, read, update and delete operations as REST endpoints that exchange JSON. MySQL stores incidents, analysis results, response actions, users and logs.

The web front end provides user sign-in and account creation, the incident reporting form, a searchable incident list with severity and status filters, a dashboard with summary cards, a seven-day trend, severity and attack-type charts, a list of open critical incidents, a 3D globe that plots geolocated public source addresses, and a reports page with time filters and CSV export. The interface indicates when the AI engine is online and shows notifications for new incidents.

4.2.2.2 Implementation Status

Table 4.3 Implementation status

Component	Status	Evidence
Spring Boot project and MySQL database	Completed	Application starts and connects to MySQL
Incident entity, repository, service, controller	Completed	Incidents stored and listed (Figure 4.14)
CRUD REST APIs	Completed	All four operations tested in Postman (Table 4.4)
User sign-in and account creation	Completed	Sign-in page (Figure 4.9)
AI classification and priority scoring	Implemented	Severity and priority shown per incident (Figure 4.14)
Response recommendations	Implemented	Recommendation stage in the reporting workflow (Figure 4.13)
Dashboard, analytics and CSV export	Implemented	Figures 4.10 to 4.12 and 4.15
Quantitative evaluation of AI accuracy	Pending	Labelled scenario testing planned

4.2.2.3 API Testing

The four CRUD endpoints were tested with Postman. Table 4.4 lists the test cases and their results.

Table 4.4 API test cases executed in Postman

Test	Operation	Expected outcome	Result
TC1	Create (POST)	New incident saved in MySQL	Pass
TC2	Read (GET)	Stored incident returned as JSON	Pass
TC3	Update (PUT)	Incident fields updated	Pass
TC4	Delete (DELETE)	Incident removed	Pass

4.2.2.4 Incident Analysis Results

Nine incidents were recorded during testing. Table 4.5 lists them as they appear in the incident list, and Figures 4.6 to 4.8 summarize them by attack type, severity and day.

Table 4.5 Incidents recorded during testing

ID	Source IP	Attack type	Target	Severity	Priority
#1	10.0.0.5	Brute Force	Admin Account	Critical	N/A
#2	10.0.0.5	Brute Force	Admin Account	Critical	N/A
#3	10.0.0.5	Brute Force	Admin Account	Critical	N/A
#4	10.0.0.5	Brute Force	Admin Account	Critical	Critical
#5	192.168.1.50	Malware	Database Server	Medium	Medium
#6	192.168.1.50	Malware	Database Server	Medium	Medium
#7	192.168.1.50	Malware	Database Server	Medium	Medium
#8	208.67.222.222	DDoS	Public Website	Medium	Medium
#9	9.9.9.9	Port Scan	Office Router	Medium	Medium

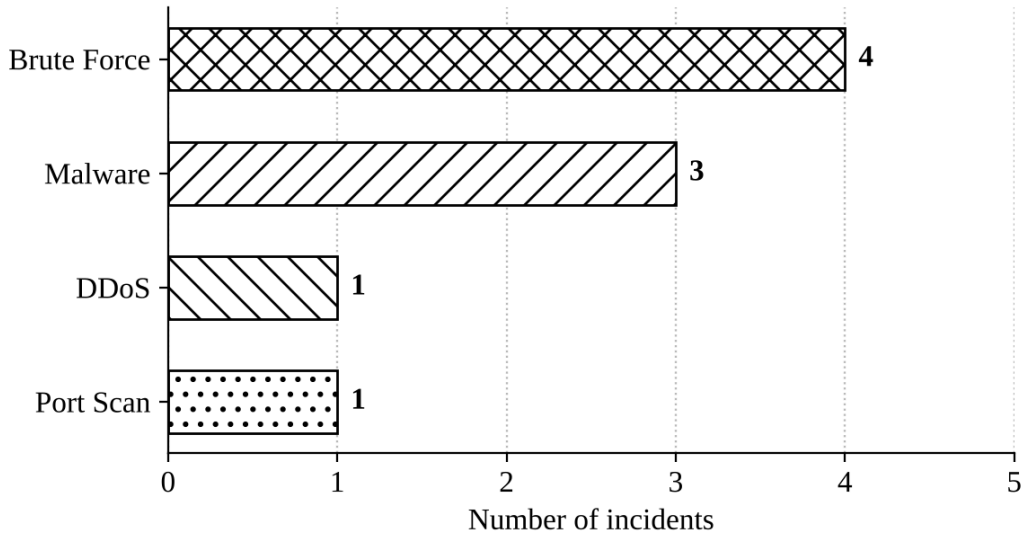


Figure 4.6 Incidents by attack type in the test data (Module 4: Dashboard and Integration)

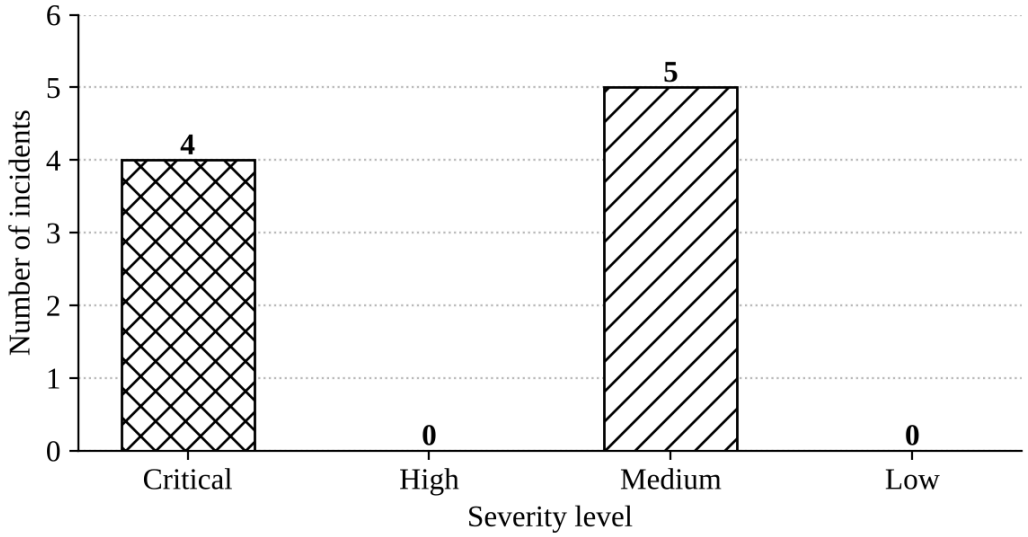


Figure 4.7 Incidents by severity level assigned by Module 3 (Intelligent Prioritization and Response)

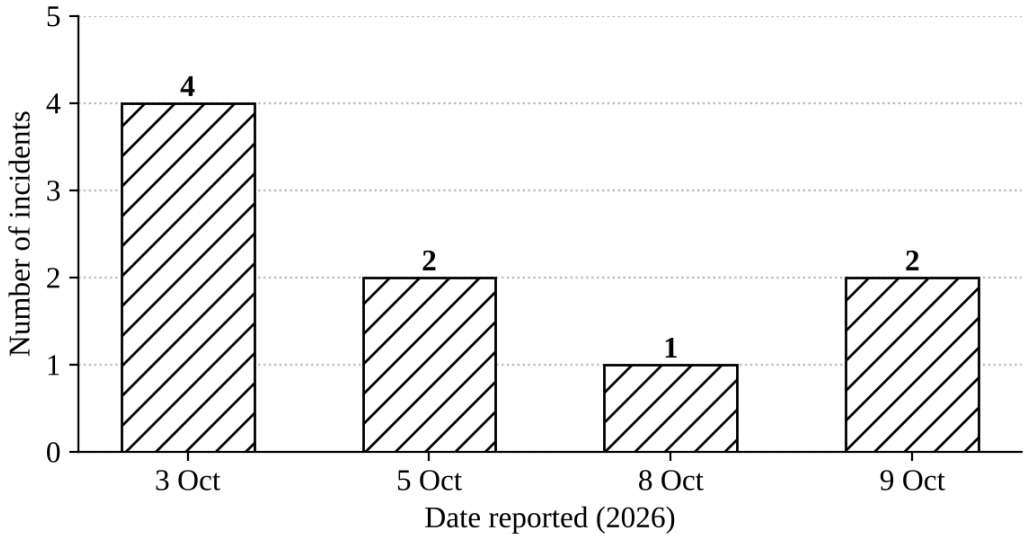


Figure 4.8 Incidents reported per day in the test data (Module 4 reports)

4.2.2.5 Summary Metrics

Table 4.6 Summary metrics reported by the system

Metric	Value	Observation
Total incidents	9	4 brute force, 3 malware, 1 DDoS, 1 port scan
Critical incidents	4	All brute-force attempts on the admin account
Average priority score	47.5 / 100	Shown as 48 on the dashboard (rounded)
Open incidents	9	No incident has been closed yet

Metric	Value	Observation
Resolution rate	0%	Follows from all incidents being open
Attack origins plotted	1	7 private or local IPs are not geolocated; 1 address could not be located

4.2.2.6 Discussion

The brute-force attempts from 10.0.0.5 against the admin account were rated Critical, which matches the rule that escalates attacks on admin and root targets. Malware traffic to the database server, the DDoS against the public website and the port scan of the office router were rated Medium. The malware incidents stayed at Medium even though the target contains the keyword database, so the keyword floor for malware on database assets should be reviewed with analysts when the thresholds are calibrated.

Incidents #1 to #3 have the same attributes as #4 but show no priority value. They were recorded before priority scoring was applied and have not been re-scored, so a re-analysis operation for existing records is planned. The 0% resolution rate reflects that the resolution workflow has not yet been used to close incidents; it is not a failure of analysis. The evaluation is preliminary: the test set has nine incidents and no independent ground-truth labels, so accuracy, precision and recall are not yet reported.

4.2.2.7 Comparison with the Existing Approach

Table 4.7 Comparison of the existing approach and the proposed system

Aspect	Existing approach	Proposed system
Incident storage	Scattered across tools	One MySQL store with REST access
Classification	Analyst reads each alert	Rule-based classification at submission
Prioritization	Analyst judgement	Heuristic score from 0 to 100 with severity levels
Consistency	Varies between analysts	Same facts always give the same result
Explanation	Rarely recorded	Rule trace stored with each analysis
Response advice	Expertise-dependent	Constraint-based recommendations
Training data	Needed for ML detectors	Not needed

In the manual workflow each of the nine incidents would require an analyst to read, rank and answer it. In the proposed system the classification, severity, priority and recommended actions are produced when the incident is submitted, and the four Critical incidents appear at the top of the Needs Attention panel.

4.2.2.8 User Interface

Figures 4.9 to 4.15 show the working application. The screenshots are reproduced in colour as captured.

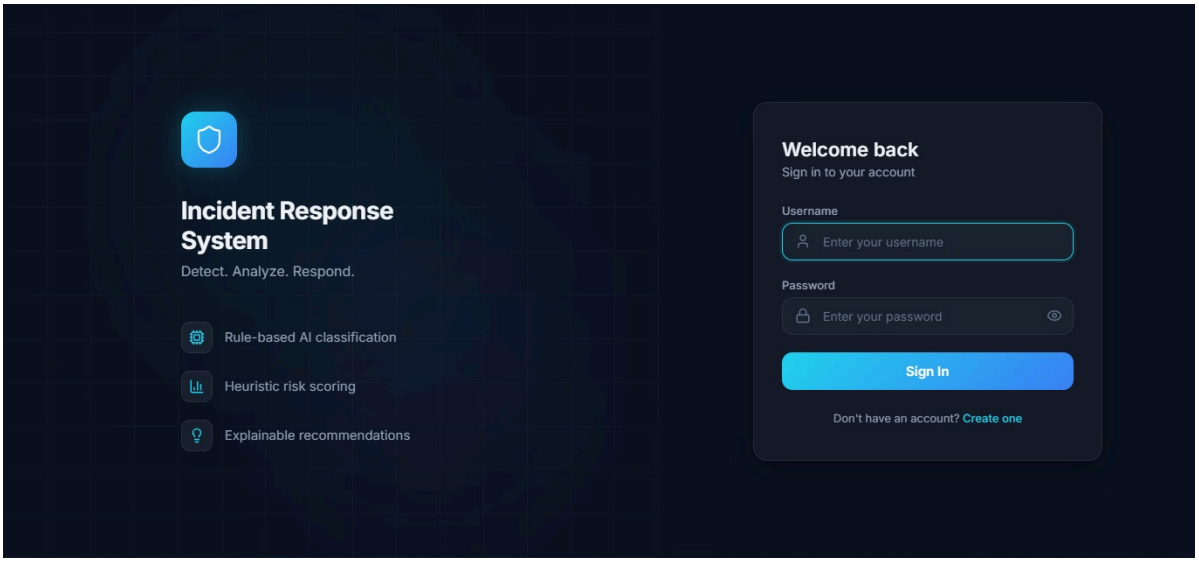


Figure 4.9 Sign-in page of the AI-Powered Incident Response System (Module 4: Dashboard and Integration)

The sign-in page states the three functions of the system, which are rule-based AI classification, heuristic risk scoring and explainable recommendations. A username and password are required before any incident data is shown, and new users can create an account.

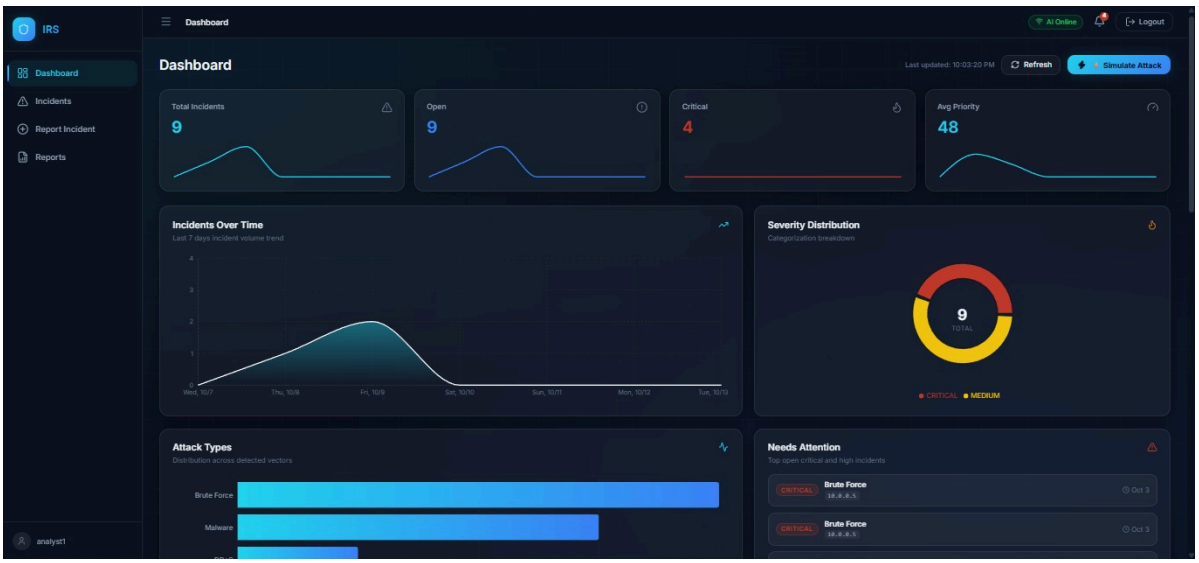


Figure 4.10 Dashboard overview with summary cards, incidents over time and severity distribution (Module 4)

The dashboard opens with four summary cards (9 total, 9 open, 4 critical, average priority 48), a seven-day trend of incident volume and a severity distribution chart. Refresh and Simulate Attack controls are placed at the top right.

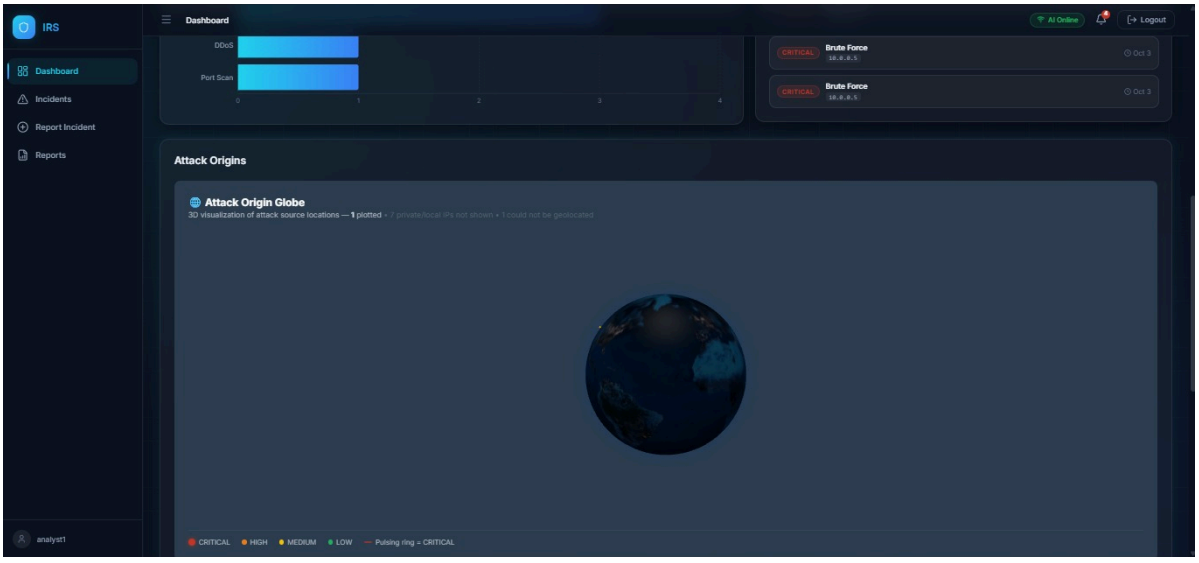


Figure 4.11 Dashboard attack types, Needs Attention list and attack-origin globe (Module 4)

The attack-type chart ranks Brute Force highest, followed by Malware, DDoS and Port Scan. The Needs Attention panel lists open Critical incidents, and the 3D globe plots geolocated public source addresses, with Critical sources marked by a pulsing ring.

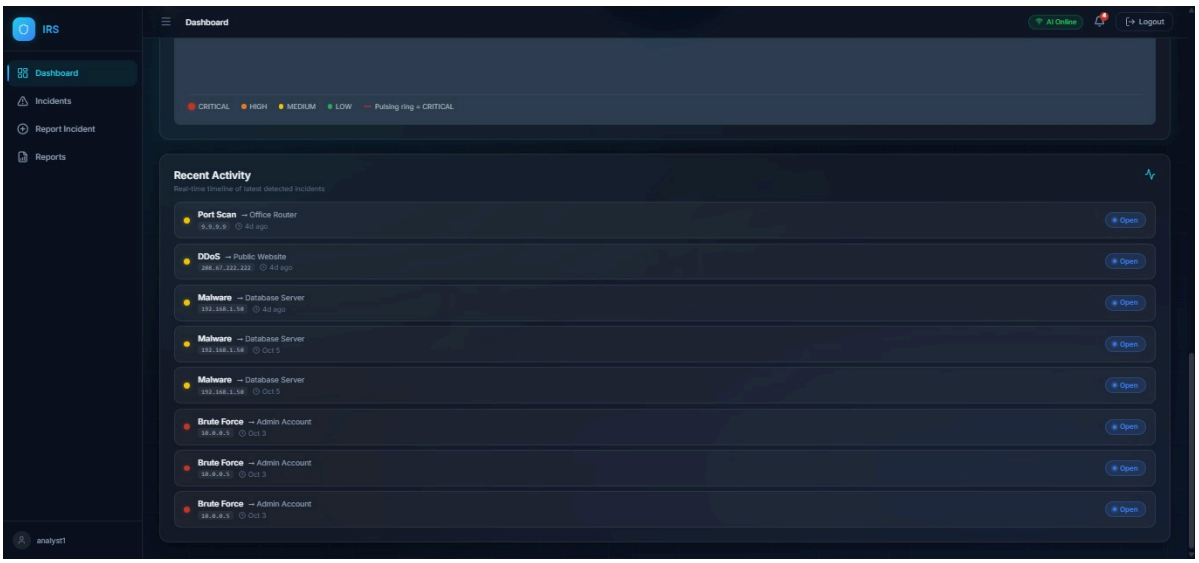


Figure 4.12 Dashboard recent activity timeline (Module 4)

The recent activity timeline lists the latest incidents with attack type, target, source IP, time and status, marked by severity.

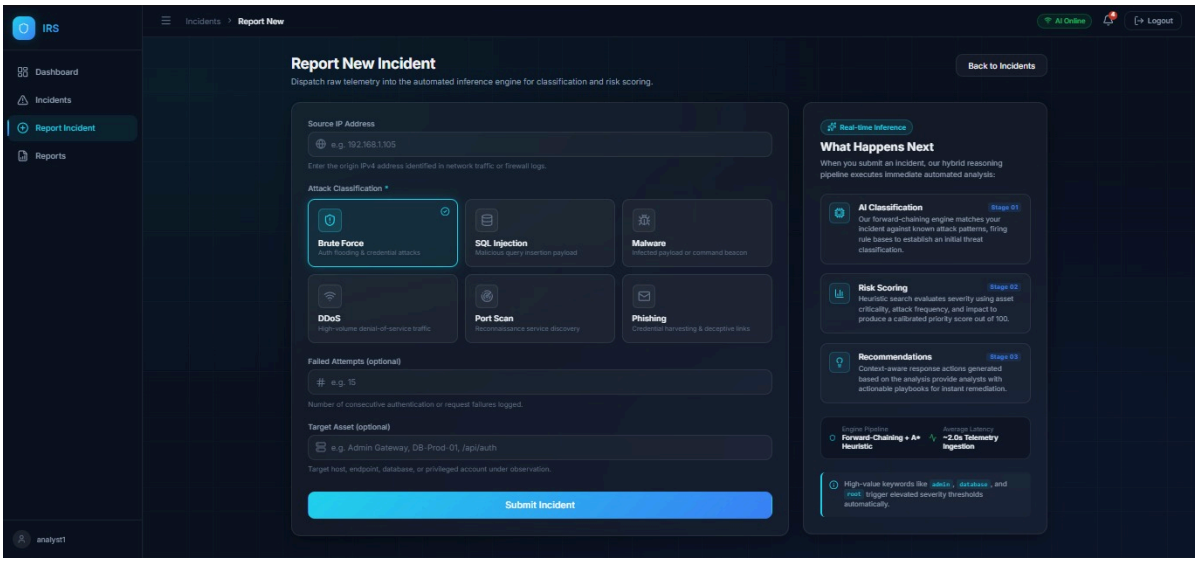


Figure 4.13 Report New Incident form feeding the analysis pipeline (Module 1 input with Modules 2 and 3)

The reporting form collects the source IP, one of six attack classifications, the number of failed attempts and the target asset. The side panel explains the three analysis stages run on submission: AI classification by forward chaining, risk scoring by heuristic search and recommendations.

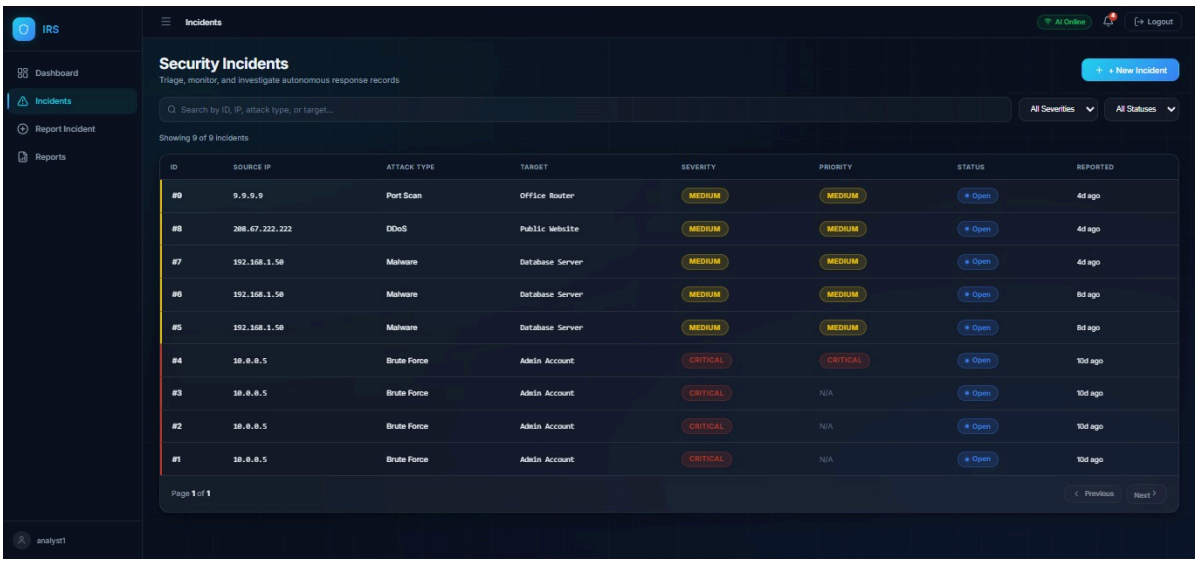


Figure 4.14 Security incidents list with severity, priority and status (Module 1 records with Module 3 output)

The incident list supports search by ID, IP, attack type or target and filtering by severity and status. Each row shows the AI-assigned severity and priority.



Figure 4.15 Reports and analytics page (Module 4: Dashboard and Integration)

The reports page shows total incidents, the average priority score (47.5 out of 100), the resolution rate and the number of critical incidents for 7-day, 30-day or all-time windows, together with charts for incidents per day, attack category, severity and response status, and a CSV export.

CHAPTER 5
CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

The **AI-Powered Incident Response System** automates cybersecurity incident analysis and response recommendation by combining cybersecurity practice, artificial intelligence, backend development and database management. The system applies rule-based reasoning, forward chaining, heuristic evaluation and constraint satisfaction to classify incidents, determine their priority and recommend suitable responses.

The implementation delivers a Spring Boot and MySQL backend with Postman-tested CRUD APIs, user sign-in, an incident reporting form, AI classification and priority scoring, and a monitoring dashboard with reports and CSV export. On the nine test incidents the system separated the four brute-force attacks on an admin account (Critical) from the five lower-risk incidents (Medium) at the moment of submission, which is the decision an analyst would otherwise make by hand. The system provides centralized monitoring and AI-assisted decision-making without needing labelled training data, and it records the rules behind every decision.

5.2 FUTURE WORK

- Measure classification accuracy and priority consistency on labelled brute-force, port-scan, DoS, malware, SQL-injection and phishing scenarios.
- Calibrate the weights and thresholds of the heuristic function with security analysts.
- Add a re-analysis action so that incidents recorded before scoring receive a priority.
- Complete the resolution workflow so that incidents can be acknowledged and closed, giving a meaningful resolution rate.
- Harden security with stricter input validation, role-based authorization tests and API request limits.
- Accept incidents automatically from firewall and IDS logs, and map incidents to ATT&CK techniques [4].

5.3 REFERENCES

[1] P. Cichonski, T. Millar, T. Grance and K. Scarfone, "Computer Security Incident Handling Guide," NIST Special Publication 800-61 Rev. 2, National Institute of Standards and Technology, Aug. 2012.

[2] K. Scarfone and P. Mell, "Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST Special Publication 800-94, National Institute of Standards and Technology, Feb. 2007.

[3] E. M. Hutchins, M. J. Cloppert and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains," Leading Issues in Information Warfare and Security Research, vol. 1, 2011.

[4] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington and C. B. Thomas, "MITRE ATT&CK: Design and Philosophy," Technical Report, The MITRE Corporation, 2018.

[5] S. J. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[6] J. C. Giarratano and G. D. Riley, Expert Systems: Principles and Programming, 4th ed. Boston, MA, USA: Thomson Course Technology, 2005.

[7] ISO/IEC 27035-1:2016, Information technology - Security techniques - Information security incident management - Part 1: Principles of incident management, International Organization for Standardization, 2016.

[8] D. E. Denning, "An intrusion-detection model," IEEE Transactions on Software Engineering, vol. SE-13, no. 2, pp. 222-232, Feb. 1987.

[9] M. Roesch, "Snort: Lightweight intrusion detection for networks," in Proc. 13th USENIX Conference on System Administration (LISA), 1999, pp. 229-238.

[10] V. Paxson, "Bro: A system for detecting network intruders in real-time," Computer Networks, vol. 31, no. 23-24, pp. 2435-2463, 1999.

[11] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," ACM Transactions on Information and System Security, vol. 3, no. 3, pp. 186-205, 2000.

[12] R. P. Lippmann et al., "Evaluating intrusion detection systems: The 1998 DARPA off-line intrusion detection evaluation," in Proc. DARPA Information Survivability Conference and Exposition (DISCEX), vol. 2, 2000, pp. 12-26.

[13]P. Garcia-Teodoro, J. Diaz-Verdejo, G. Macia-Fernandez and E. Vazquez, "Anomaly-based network intrusion detection: Techniques, systems and challenges," Computers & Security, vol. 28, no. 1-2, pp. 18-28, 2009.

[14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symposium on Security and Privacy, 2010, pp. 305-316.

[15] M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA), 2009, pp. 1-6.

[16] H.-J. Liao, C.-H. R. Lin, Y.-C. Lin and K.-Y. Tung, "Intrusion detection system: A comprehensive review," Journal of Network and Computer Applications, vol. 36, no. 1, pp. 16-24, 2013.

[17] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," IEEE Communications Surveys & Tutorials, vol. 18, no. 2, pp. 1153-1176, 2016.

[18] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108-116.

[19] A. Khraisat, I. Gondal, P. Vamplew and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," Cybersecurity, vol. 2, art. 20, 2019.

[20] C. L. Forgy, "Rete: A fast algorithm for the many pattern/many object pattern match problem," Artificial Intelligence, vol. 19, no. 1, pp. 17-37, 1982.

[21] P. Mell, K. Scarfone and S. Romanosky, "A Complete Guide to the Common Vulnerability Scoring System Version 2.0," Forum of Incident Response and Security Teams (FIRST), Jun. 2007.

[22] P. E. Hart, N. J. Nilsson and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, Jul. 1968.

[23] A. K. Mackworth, "Consistency in networks of relations," Artificial Intelligence, vol. 8, no. 1, pp. 99-118, 1977.

[24] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[25] OWASP Foundation, "OWASP Top 10: 2021," Open Worldwide Application Security Project, 2021.

5.4 APPENDIX

5.4.1 Appendix A: Database Schema

Table 5.1 Core entities and fields of the database schema

Entity	Fields
Incident	id (Long), title, description, incidentType, severity, status (String), createdAt, updatedAt (Date)
IncidentAnalysis	id, incidentId (Long), classification (String), priorityScore (Integer), priorityLevel, recommendedResponse (String), analysisDate (Date)
ResponseAction	id (Long), incidentType, actionDescription, constraints (String)
User	id (Long), name, email, role, password (String)
Role	id (Long), roleName, description (String)
SystemLog	id, userId (Long), action (String), timestamp (Date), details (String)

5.4.2 Appendix B: API Operations

Table 5.2 Incident CRUD operations

Operation	HTTP method	Service method	Tested
Create incident	POST	createIncident()	Yes
Read incident(s)	GET	getIncidentDetails()	Yes
Update incident	PUT	updateIncident()	Yes
Delete incident	DELETE	delete by id	Yes

5.4.3 Appendix C: Project Schedule

Table 5.3 Project schedule

Phase	Work	Status
Review 1	Problem definition, objectives, literature survey, proposed architecture and methodology	Completed
Review 2	Spring Boot backend, MySQL integration, CRUD APIs, Postman testing, DFD and UML design, threat model	Completed
Implementation	AI classification, heuristic priority scoring, recommendations, dashboard, reports and sign-in	Implemented
Final review	Accuracy evaluation, resolution workflow, security hardening and final documentation	Pending